리서치

리서치 방법론

교수나 R&D 엔지니어들도 “감으로” 설계하는 게 아니라, 결국 몇 가지 정해진 채널을 반복해서 쓰면서 문헌·공식 자료를 모아온다. 네가 지금부터 익숙해지면 되는 것들만 정리해줄게.

---

1. 논문·학술 자료 찾을 때 쓰는 사이트

- Google Scholar
  - 공대 쪽에선 거의 기본 출발점.sciencedirect
  - 키워드 예:
    - “driver monitoring system distraction warning”,
    - “minimal risk manoeuvre R171”,
    - “container-based fail-operational SDV architecture”.
- IEEE Xplore
  - 자율주행, 임베디드, RTOS, SDV, 차량 SW 논문 대부분이 여기에 있음 (IV, ITSC, RTSS, RTAS, DATE 등).guides.loc
- ScienceDirect / Springer / ACM Digital Library
  - 실시간 시스템, HMI, 인간-기계 인터랙션, DMS 관련 논문도 여기 많이 있다.reddit+1

학교 도서관 통해 접속하면 대부분 무료로 볼 수 있고, 엔지니어들도 실제로 이 세트리오**(IEEE + Springer/Elsevier + ACM)**를 제일 많이 쓴다.reddit+1

---

2. “시스템 설계에 바로 도움되는” 자료들

교수·산업 연구자가 시스템 설계할 땐, 순수 논문만 보는 게 아니라:

- 표준/규격
  - UNECE R171, R157, ISO 26262, ISO 21434 같은 국제 표준 문서.
  - 이런 건 규격 사이트나, 요약한 기술 블로그/화이트페이퍼 통해 많이 본다.sciencedirect
- 학회·워크숍 튜토리얼, survey 논문
  - “Digital engineering state-of-the-art”, “Driver monitoring systems – trends and developments” 같은 종합 리뷰/튜토리얼 논문으로 전체 지형을 먼저 본다.seeingmachines+1
- 회사/연구소 기술 리포트
  - Bosch, Mobileye, NVIDIA, OEM(현대차 등)이 내는 **whitepaper, tech blog**는 실제 시스템 구조·파라미터 감 잡는 데 많이 쓰인다.developer.nvidia+1

실제 패턴은:

1. Google Scholar/IEEE로 키워드 검색
2. survey/overview/standard 문서로 큰 줄기 파악
3. 구체 구현이 필요하면 individual 논문·화이트페이퍼·레퍼런스 설계 참고

이런 식이다.dev+1

---

3. 네가 지금 해야 할 리서치 루틴 예시

너 DCAS/DMS 시스템 설계 기준으로라면, 루틴을 이렇게 잡으면 된다:

- 도메인 키워드 세트 정하기
  - 예: {“driver monitoring distraction warning”, “minimum risk manoeuvre highway”, “fail-operational SDV container”, “real-time multi-DNN inference automotive”}
- 각 키워드를
  - Google Scholar → 관련 논문 리스트 확보sciencedirect
  - IEEE Xplore → 자동차/임베디드 쪽 학회 논문 확인guides.loc
    - UNECE/ISO 규격 텍스트(이미 R171 보고 있으니까 계속 확장)

그리고 논문 읽을 때도:

- 개별 논문 하나를 처음부터 끝까지 파는 게 아니라,
  1. 제목/초록/그림/결론으로 “우리 시스템 설계에 쓸 수 있는 포인트가 있나?”만 체크
  2. 쓸 수 있으면 그 부분(아키텍처 그림, 상태 정의, 타이밍 수치)만 자세히 읽고 메모

이게 실제 교수·연구자들이 시스템 설계할 때 쓰는 방식에 가깝다.gaudisite+1

정리하면,
“특별한 비밀 사이트”가 있는 게 아니라

- Google Scholar + IEEE Xplore + (ScienceDirect/ACM)
  - 표준/규격 사이트 + 회사/연구소 whitepaper

이 조합을 가지고, 지금 네가 하는 것처럼 **“문제 정의 → 관련 키워드 → 논문/규격 검색 → 설계에 맞게 추려서 쓰는”** 루프를 계속 도는 게 교수·R&D 엔지니어들의 기본 패턴이다.sciencedirect+1

IdeaV1. R157

RTOS in Container

리서치

1. RTOS in Container란?

- **<u>RTOS</u>**: "작은 가상 ECU 같은 RTOS를 컨테이너/프로세스로 하나 더 올리고, 이걸 Linux와 나란히 돌리면서 통신"
- <u>왜 필요한거야**?**</u>
  - **1.** 더 큰 관점은 **"혼합 크리티컬리티(mixed-criticality)"** 구조다**.**
    - 말 그대로 "중요도가 다른 기능들이 한 HW 위에 같이 있는 상태"를 말한다.
      - 예: 에어백, 브레이크 제어 같은 건 "실패하면 사람 죽을 수 있음" → 최고 크리티컬.
      - 뮤직 플레이어, 테마 변경, 지도 색깔 바꾸기 같은 건 "고장 나도 차는 굴러감" → 낮은 크리티컬.
    - 자동차에서는 이 둘이 같은 ECU/SoC 위에 공존하는 경우가 많아서,
      - 높은 크리티컬 기능은 RTOS/안전 파티션 쪽으로,
      - 낮은 크리티컬 기능은 Linux/컨테이너 쪽으로 분리해서 돌리자는 흐름을 "혼합 크리티컬리티 아키텍처"라고 부른다.
  - 2. 실시간성이 보장되어야할 프로세스를 관리하기 위함
- <u>어떻게**?**</u>
  - RTOS는 태스크 우선순위, 인터럽트, 스케줄링이 실시간 보장에 맞춰 설계됨.
- <u>리눅스와 뭘 통신해**?**</u>
  - Linux → RTOS
    - 상위 레벨 명령: "목표 속도 5km/h로 주차 모드 유지", "조향 각도 제한 15도 이내", "긴급 정지 요청" 등.
    - 환경 정보: 지도/경로 계획 결과, HMI에서 온 운전자 버튼 상태(자율모드 ON/OFF) 등.
  - RTOS → Linux
    - 실시간 모니터링 결과: "현재 속도/조향/가속도 값", "센서 신뢰도", "제동 응답 지연" 등.
    - 안전 관련 이벤트: "제약 조건 위반 감지", "센서 오류", "제어 명령 거부/수정" 같은 Fail-safe/Fail-operational 신호
- <u>왜 **'in Container'**야**?**</u>
  - 개발·배포 방식 통일
    - 기존 RTOS 앱은 빌드/배포 흐름이 Linux/클라우드와 많이 달라서 DevOps/클라우드 툴이 잘 안 먹는다.

        - RTOS나 RTOS 기반 서비스들을 “컨테이너 이미지”처럼 포장하면,
            - Docker/OCI 기반 빌드·배포·업데이트(OTA) 체계와 같이 쓸 수 있다.
    - Linux/컨테이너 세계와의 인터페이스
        - Linux 쪽에서는 “그냥 또 하나의 컨테이너/서비스”처럼 RTOS 모듈을 관리하고,
        - 내부에서는 RTOS 스케줄러/태스크가 돌면서 실시간·안전 로직을 처리.
    - 컨테이너 안에 돌고 있는 건 **RTOS** 기반의 작은 앱/런타임이고**, Linux** 입장에선 그것도 또 하나의 컨테이너 서비스처럼 보이게 하자**”**라는 발상이다**.**
- **<u>Zephyr, NuttX</u>**<u>는 뭐야</u>**<u>?</u>**
    - 둘 다 오픈소스 RTOS의 종류다.
        - Zephyr:
            - Linux Foundation 주도, Intel·Nordic·NXP 등 후원.
            - 네트워킹, BLE, 파일시스템 등 기능 풍부, IoT/임베디드용 RTOS.
        - NuttX:
            - POSIX 친화적, “작은 유닉스 같은 RTOS” 지향.
    - FreeRTOS 같은 것과 같은 RTOS 계열인데, 요즘 오픈소스 커뮤니티/산업에서 주목받는 신흥 RTOS라고 보면 된다.
- **<u>AUTOSAR</u>**<u>랑 무슨 관련</u>**<u>? “</u>**<u>대신 쓸 수 있다</u>**<u>”</u>**<u>는 말의 의미</u>
    - AUTOSAR는 자동차 소프트웨어 아키텍처/플랫폼 표준이다.
        - Classic: RTOS 위에 BSW·RTE·SWC 올리는 구조.
        - Adaptive: POSIX OS(Linux/QNX) 위에 C++ 앱 올리는 구조.
    - 실제로 AUTOSAR Classic도 “RTOS + 표준화된 미들웨어/드라이버 세트”라고 볼 수 있어서,
        - 어떤 사람들은 Zephyr/NuttX 같은 RTOS에 자동차용 미들웨어를 얹어서
          “미래 AUTOSAR 대체/보완 후보”처럼 보는 시각도 있다.
    - “우리 세대엔 AUTOSAR Classic이 표준이었는데, 너희 세대엔 Zephyr/NuttX 같은 오픈소스 RTOS 위에 자동차용 스택을 얹어 쓰는 흐름도 생길 수 있다.”
        - 개념적 카테고리로 보면 “AUTOSAR Classic가 차지하던 위치에 Zephyr/NuttX 계열이 들어올 수도 있다”는 이야기.
        - 완전히 동일한 것은 아니지만, “자동차용 RTOS/미들웨어 플랫폼”이라는 관점에서 같은 경쟁구도 안에 있는 셈.

- **<u>RTOS</u>**<u>를 자율주행과 엮을 수 있나</u>**<u>?</u>**

    - “LKAS/자율주차 기능 위에 RTOS 안전 모듈을 하나 더 얹고, 조건 위반 시 Linux/HU에 경고·모드 전환 요구를 알려주는 구조”로 가져가면,
    - 발표용 스토리도 설득력 있고, 산업적으로도 말이 된다 (MRM 개념 언급해주면 더 좋음).
- <u>갑자기 수동 모드로 던지면 안되잖아**?**</u>
    - “Minimal Risk Maneuver (MRM, 최소 위험 상태로 이동)” 개념이 있다.
    - 즉, 문제가 나면 그냥 “바로 수동으로 던지는 게 아니라”,
        - 속도를 줄이고,
        - 차선 유지/비상등 같은 걸 유지하면서,
        - 운전자가 받을 준비를 했는지/못하면 차량 스스로 정차하는 전략을 사용.
- <u>자율주행 팀원에게 도움이 되나**?**</u>
    - 자율주행 팀 포트폴리오에 좋은 포인트:
        - “우리는 LKAS/자율주차 기능만 만든 게 아니라, 시스템이 이상 상태를 감지했을 때 Minimal Risk Maneuver 시나리오까지 설계/구현했다.”
        - 예:
            - LKAS/자율주행 중 오류 감지 →
            - 속도 서서히 감소, 차선 유지 또는 안전한 위치로 정차 →
            - 운전자에게 수동 인계 안내.
        - 이런 스토리는 ADAS/AV 쪽에서 실제로 쓰는 개념이라, 이력서/발표에서 꽤 먹힌다.

- <u>역할 분담**?**</u>
    - 사람 A: **Jetson OTA** 담당
        - Jetson L4T/Yocto 기반 AB 파티션, SWUpdate·RAUC 중 하나 선택해서 OS OTA 구현.NEXT_PROJECT_ANALYSIS.md+1
        - 필요하면 RPi 쪽 VehicleControlECU 바이너리 OTA까지 확장.
        - 발표에서 “SDV에서 OTA가 어떻게 돌아가는지” 축 담당.
    - 너: **RTOS +** 전체 통합 담당
        - Zephyr/NuttX 기반 안전 모듈(실시간 상태 모니터 + 조건 위반 이벤트) 설계·구현.
        - Jetson·RPi·RTOS·LKAS 모듈·OTA를 vsomeip/네트워크로 한 구조 안에 묶는 아키텍처·서비스 통합.
        - HU/IC HMI에서 자율주행/Fail-safe 상태를 보여주는 부분까지 총괄.ONBOARDING_GUIDE.md+1
    - 사람 B(자율주행 팀원): **LKAS + MRM/**자율주차 로직 담당
        - LKAS 모듈, 후방 주차 앱과 연계한 자율주차 제어.

    - RTOS에서 오는 Fail-safe 이벤트를 입력으로 받는 MRM(감속·정차·수동 인계) 로직 구현.

이렇게 하면:

- 프로젝트 스토리: **"SDV 플랫폼 + OTA + RTOS 안전 모듈 + LKAS/자율주차"**를 하나로 엮을 수 있고,
- 네 포지션: 플랫폼·통합 + **RTOS**·혼합 크리티컬리티 두 축을 동시에 가져가는 구조라 취준용으로도 가장 좋다.

RTOS in container가 핫한 이유?

"요즘은 아예 하나의 컴퓨터에 차량제어 로직도 다 들어간다" → 그래서 RTOS in Container같은게 핫하다. → 실제로 하나의 SoC에 제어로직 때려 넣음?

- 광의의 'vehicle computer'/도메인 컨트롤러 안에 많은 제어 로직이 통합되는 건 맞지만,
- 브레이크·조향 같은 핵심 안전 기능을 인포테인먼트용 SoC 하나에 다 때려박는 수준은 아니다. 여전히 ECU/MCU·도메인 컨트롤러 구조가 남아 있다

- "그래서 우리처럼 Jetson(상위) + VCP-G MCU(제어)를 물리적으로 나누는 구조가 시대에 뒤떨어졌냐?" →
    - 아니다. 이건 여전히 ISO 26262 관점에서 굉장히 보수적이고 자연스러운 구조고,
    - OEM들이 가는 중앙집중형 SDV도 결국 "중앙 컴퓨터 + 주변의 안전/액추에이터 ECU"라는 큰 틀 안에 있다.

- 공개 자료(현대 E-GMP, PBV 등)는 파워 일렉트로닉스·모터 인버터·샤시 통합 제어를 강조하지, "인포테인먼트/ADAS용 vehicle computer에 엔진/브레이크 제어까지 다 올렸다"라고 말하진 않는다.
- 테슬라도 "중앙집중형 아키텍처, 적은 수의 강력한 ECU"를 지향하지만,
    - 브레이크, 에어백, 스티어링 등은 규제·안전 때문에 여전히 전용 안전 ECU/MCU가 있고,
    - 인포테인먼트/Autopilot 컴퓨터는 그 위에서 ADAS·경로계획·HMI를 담당하는 구조로 보는 게 업계 일반적인 이해다

https://www.linkedin.com/pulse/vehicle-ee-architecture-explained-domain-vs-zonal-designs-dewangan-43ssf/

3/17

1. 내가 속한 팀은 임베디드 시스템 팀이고 총 2명이야. 우리팀 상황: 젯슨 오린 나노를 HPC, rasberrypi를 zonal ECU로 사용하며 이더넷 케이블로 연결됨. 젯슨에는 인포테인먼트 앱들이 도커 컨테이너 안에서 돌아가고 있음. 현재 OTA를 막 시작함.

2. 이시우네 팀은 ADAS팀이고 현재 상황은 아래와 같음.

현재 JetRacer 자율주행 프로젝트로 딥러닝 기반 차선 인식 및 steering 제어, YOLOv8과 Depth 카메라를 활용한 객체 인식과 거리 측정, 그리고 rule-based 장애물 회피 시스템까지 구현을 마쳤습니다. 현재는 수동 주행 데이터를 수집해 neural network 기반 장애물 회피로 전환하는 작업을 진행 중입니다.

3. 우리팀과 이시우네 팀은 합칠 예정임. 왜냐하면 이시우네 팀원이 취업 이슈로 다 나갔고, 남은 한명도 나갈 가능성이 높아서 파이널 프로덱투를 혼자허기 어렵다고함.

4. 합쳐서 총 셋이서 진행할 최종 푸로젝트 주제는 RTOS In container임. 구상 중인 최종 데모 시나리오는 아래와 같음. 교수님께서 방향성 괜찮다고 하시면 구체적인 사항 걀정할 생각임.

1. 정상: LKAS/자율주차 ON, HU에 상태 표시.
2. OTA(조작된 업데이트) → 이상 명령.
3. RTOS(Zephyr/NuttX)로 Linux 제어 명령을 실시간으로 검증하고, 위배 시 이벤트 발행 → Minimal Risk Maneuver 트리거.
4. HU: “안전 제한, 수동 전환” 표시 + 로그.

교수님 미팅 준비 3/27

- [x] ~~텔레칩스 각 모델별 활용 가능성 조사~~
- [ ] 우리 아이디어 구체화
    - [ ] MCU가 어떻게 심각한 이상을 탐지?
    - [ ] MCU가 어떻게 MRM을 수행?
    - 시나리오
        - MCU 사용 → RTOS in Container 사용 불필요
        - MCU 사용X → 라즈베리파이에 RTOS in Container 사용
    - [ ] ISO 26262, 오토사 등 합리화.
- [ ] coretex
- [x] ~~우리 아이디어 + VCP-G 보드(FreeRTOS를 지원하고, 향후 업데이트에서는 ZephyrOS도 지원할 예정)~~
- [ ] 교수님 보여드릴 우리팀 깃허브
- [ ] 교수님 보여드릴 PPT만들기

모델별 활용 가능성 조사

1) D3-G (Telechips Dolphin3 SoC 기반 SBC)

- 장점
    - 자동차용 Telechips Dolphin3 SoC(Cortex-A72/A53 + R5) 기반, 3x CAN, PCIe, CSI 등 실제 차량용 **HPC**에 가까운 하드웨어 구성.
    - Yocto Linux SDK 제공, Ubuntu 22.04 rootfs 지원 → "자동차용 HPC + Yocto 경험"이라는 학습/브랜드 가치를 가질 수 있음.topst+1
- 단점
    - **Jetson Orin Nano**와 역할이 상당 부분 겹침(ADAS/HMI/HPC 역할).
    - BSP/툴체인, Yocto 빌드 환경을 새로 구축해야 하고, Telechips SDK는 Linux 5.10 기반이라 학기 내에 안정적으로 포팅·디버깅까지 하기엔 부담.
    - GPU·멀티디스플레이 등은 IVI/콕핏엔 좋지만, 이번 프로젝트 핵심인 "RTOS+MRM"과 직접 연결되지는 않음.
- 결론
    - 시간·인력 여유가 있다면 "자동차용 HPC 보드 경험"을 위해 서브로 써볼 수는 있지만, 현재 일정 기준으로는 Jetson을 이미 쓰고 있어서 필수 도입까지는 필요 없다고 판단.

---

2) AI-G (Telechips N-Dolphin 기반 NPU 보드)

- 장점
    - Telechips N-Dolphin SoC 기반 **NPU** 보드, 8 TOPS급 NPU와 Cortex-A53 쿼드 코어 탑재 → 추후 JetRacer 비전/ADAS 가속기 용도로 연구해볼 수 있는 플랫폼.
- 단점

    - 현재 **ADAS** 팀 코드가 **CUDA/Jetson** 생태계를 기준으로 작성되어 있음.
    - 학기 중에 NPU 기반으로 전체 ADAS 파이프라인을 이식·튜닝하는 것은 리스크가 크고, 우리가 이번에 집중하려는 "RTOS+MCU+MRM 구조 설계"와는 초점이 다름.
- 결론
    - 장기적으로 **Telechips NPU** 생태계 경험을 쌓기 위한 후보는 될 수 있으나,
    - 이번 학기 프로젝트 구조/일정 기준에서는 현실적인 도입 대상이 아님.

---

3) VCP-G (Cortex-R5F MCU 보드)

- 장점
    - Arm Cortex-R5F 200MHz, 4MB Flash, CAN/CAN-FD, UART 등 자동차용 실시간 제어에 적합한 MCU.
    - FreeRTOS 공식 지원, Zephyr 지원 예정, Arduino 호환 → 우리가 설계한 "RTOS 기반 안전 모듈 + MRM" 구조를 실제 자동차용 MCU 위에서 검증하기에 가장 적합.
    - Jetson(ADAS/HMI) + RPi(Zonal/Gateway) + VCP-G(MCU) 3단 구조로, HPC + Zonal + MCU 아키텍처 교육·연구용 데모를 만들 수 있음.
- 단점
    - 팀 내 RTOS 경험이 적다면 초기 러닝 커브가 존재. 다만 범위를 "안전 감시 + MRM"으로 제한하면 학기 내 구현 가능성이 있음.
- 결론
    - TOPST 플랫폼을 활용하면서도, 우리가 원래 기획했던 **"MCU + RTOS + MRM"** 아이디어에 가장 잘 부합하는 선택지.
    - Jetson 기반 기존 구조를 유지하면서 VCP-G를 추가하는 방향이, 일정·위험·교육 효과 측면에서 가장 합리적이라고 판단.

VCP-G활용한 우리 아이디어 구체화

1. 이것을 하드웨어적으로 어떻게 연결? HPC + Zonal + MCU 구조? 아니면 라즈베리파이 대체? → 차와 라즈베리파이 사이에.
2. 소프트웨어적으로는 무슨 역할? → 자율주행 모듈 감시자 + MRM 로직.
3. 제퍼는 언제 사용가능?
    a. 정확히 명시된 것 없음. 메일로 문의했음. → 아직 정해진 바가 없다고 하심.
4. Free RTOS Vs. zephyr

젯슨 - RPI - MCU - 차량 흐름 검토

1. 현재 제어 흐름 (Raspberry Pi → 검은 판떼기)
사용자님이 나열하신 핀(SCL, SDA, GND, 3V3, 5V)은 전형적인 **I2C** 통신 및 전원 공급 라인입니다. 현재 시스템의 데이터와 전력 흐름은 다음과 같습니다.

A. 데이터 흐름 **(Control Logic)**

1. 명령 생성 **(RPi):** 라즈베리 파이의 CPU가 파이썬 코드를 실행하여 "조향 15도, 속도 50%"라는 데이터를 생성합니다.
2. **I2C** 직렬 통신**:** 이 데이터는 **SDA(데이터)**와 **SCL(**클럭**)** 핀을 통해 직렬 패킷 형태로 검은 판떼기로 전송됩니다.
3. **PWM** 변환 **(PCA9685):** 판떼기에 있는 PCA9685 칩이 I2C 신호를 받아 각 장치에 맞는 **PWM(**펄스 폭 변조**)** 신호를 생성합니다.
    - 조향 서보모터**:** 20ms 주기의 PWM 신호 중 펄스 폭(약 1~2ms)에 따라 각도를 조절합니다.
    - 구동 모터 **(2**개**):** 모터 드라이버(TB6612)로 PWM 신호를 보내 듀티 사이클(Duty Cycle)에 따라 모터에 가해지는 전압을 조절해 속도를 제어합니다.

B. 전력 흐름 **(Power Management)**

1. 배터리**(12.6V):** 판떼기의 전원 단자에 연결되어 모터 드라이버에 직접 전력을 공급합니다.
2. **5V/3.3V** 전압 강하**:** 판떼기 내부의 레귤레이터가 배터리 전압을 낮춰 라즈베리 파이(5V)와 논리 회로(3.3V)에 전원을 공급합니다.

---

2. 현재 상태 vs MCU 삽입 상태의 상세 비교
이 부분이 SEA:ME 과정에서 가장 중요하게 다루는 **'실시간성'**과 **'안전'**의 차이입니다.

비교 항목	현재 상태 **(RPi** 단독**)**	**MCU** 삽입 상태 **(RPi + MCU)**
시스템 구조	중앙 집중형 **(Single-tier)**	분산 제어형 **(Multi-tier / Automotive)**
제어 주체	라즈베리 파이 (Linux OS)	**MCU (RTOS** 또는 **Bare-metal)**

실시간 성능	비결정적 **(Jitter** 발생**).** 카메라 연산이 많아지면 모터 제어 신호가 미세하게 밀릴 수 있음.	결정적 **(Precise).** 센서 융합 및 모터 제어 루프를 마이크로초 단위로 일정하게 유지.
안전 **(Safety)**	소프트웨어 버그나 OS 다운 시 차량이 폭주할 위험이 큼.	**Fail-safe.** RPi가 멈춰도 MCU가 이를 감지해 즉시 차량을 정지(E-Stop) 시킴.
통신 프로토콜	I2C (근거리, 노이즈에 취약)	**CAN** 또는 **UART.** 차량 통신 표준에 더 가까운 방식 사용 가능.

🔍 비판적 분석: 왜 MCU를 넣어야 하는가?

1. **OS**의 한계**:** 라즈베리 파이에서 돌아가는 표준 Linux는 **RTOS(Real-Time OS)**가 아닙니다. 멀티태스킹 과정에서 CPU가 AI 연산에 집중하느라 PWM 명령 전송을 아주 잠깐(예: 10ms)만 늦춰도, 고속 주행 중인 차량은 이미 경로를 이탈하게 됩니다.
2. 추상화와 격리**:** 현대 자동차는 인포테인먼트(라즈베리 파이 역할)와 주행 제어(MCU 역할)를 엄격히 분리합니다. MCU를 넣는 것은 **"**인식**(Perception)"**과 **"**제어**(Actuation)"**를 하드웨어적으로 격리하여 시스템의 신뢰성을 확보하는 연습입니다.
3. **Zephyr OS** 활용**:** 나중에 Zephyr OS를 공부하실 예정이라면, MCU에서 Zephyr의 타이머와 인터럽트를 활용해 모터를 제어하고 라즈베리 파이와는 통신(UART/CAN)만 주고받는 구조를 만드는 것이 이 프로젝트의 정점(Apex)이 될 것입니다.

문제 상황 시나리오

자율주행 팀의 한계점을 보완하는 흐름으로 시나리오 구상하는게 어떨까?
→ ADAS팀의 특징을 파악하고 그 특징의 한계를 조사

1. 카메라 기반
    a. 날씨 영향 → LKAS 작동 안함. → 수동운전!!
2. (현재 시우 완성)룰기반
    a. 룰에 기재되지 않은 상황 발생 → 수동운전!!
3. (동현이형 완성 시)딥러닝 기반
    a. 처음 마주한 상황 → DL의 솔루션에 확신이 없음 → 수동운전!! → 차 자체가 컴퓨터이므로 실시간으로 데이터 학습 → 최소 3번 학습 → 자율주행 가능
    b. 임베디드팀: 장애물 앞에서 RTOS 있을 때, 없을 때 반응 속도 비교.

구체적 구현 방안

1. 옵션 A vs B: 뭐가 더 바람직?

"이상적인 산업 아키텍처" 기준으론 옵션 **A**가 더 깔끔하다.

- 옵션 A
  - I2C 마스터 = MCU, PiRacer 보드는 단순 액추에이터/센서 보드.
  - Jetson → RPi(게이트웨이) → MCU → PiRacer 순서지만, 제어·센서 루프는 **MCU**에서 완결.
  - "센터(HPC)는 고수준 판단, 로컬 MCU는 센서·액추에이터 제어"라는 zonal/edge 컴퓨팅 트렌드와 잘 맞는다.
- 옵션 B
  - Pi도 I2C를 계속 쓰고, 사이에 MCU가 껴서 감시·개입.
  - 구현은 쉬운데, "제어 책임이 누구냐"가 애매해지고, 구조가 복잡해진다.

시간·난이도 안 따지고 "바람직"만 보면 옵션 **A**를 목표로 잡는 게 맞다.
현실적으로 힘들면 A를 "타깃 아키텍처", B를 "프로토타입 구현"으로 설명하는 식으로도 갈 수 있다.

---

1. MCU가 "심각한 이상"을 탐지하는 역할이 가능하냐?

가능하다. 실제로 그런 역할을 하는 **safety supervisor / watchdog MCU** 개념이 존재한다.

- 상위(ADAS/HPC)에서 오는 정보:
  - 모드 상태, confidence/진단 플래그, 명령 값, ODD 관련 플래그 등.
- 차량 상태:
  - 속도, 조향, 센서 상태, 통신 타임아웃 등.

MCU는 이걸 보고 규칙 기반으로:

- "상위 시스템 고장(severe ALKS failure)"
- "차량 상태 고장(severe vehicle failure)"

로 분류하는 룰을 돌릴 수 있다.
논문/특허에서도 "프로세서가 주변환경·차량 상태를 보고 MRM 필요 여부를 판단, 컨트롤러가 MRM 수행" 같은 구조를 그대로 쓰고 있다.

너 프로젝트에선:

- Jetson이 보내는 ADAS 상태/진단 플래그(카메라 불량, 룰 fallback, DL confidence 낮음)를 MCU가 수신.
- 여기에 속도·조향·장애물 거리·통신 타임아웃 등을 조합해 "심각한 이상"을 정의하면 된다.

---

1. MRM 때 MCU가 직접 자율주행을 하냐?

여기서 “자율주행”이라는 표현을 조심해야 한다.

- 실제 MRM 정의는 “최소 위험 상태로 이동(대개 차로 내 감속·정지)”이지, 새로운 경로 계획까지 하는 자율주행은 아님이다.
- 특허 예시도 대부분:
  - 주변 환경/차량 상태로 “MRM 필요 여부” 결정,
  - MRM 타입(직진 정지, 차로 변경 후 정지 등) 선택,
  - 가속·제동·조향을 제어해서 안전하게 멈추는 수준이다.

너 프로젝트에서는:

- 평상시:
  - ADAS(시우/동현이형 코드)가 경로/속도/조향 제안.
  - MCU는 그걸 따라가면서 제어.
- “심각한 이상” 판단 후:
  - MCU가 더 이상 **Jetson/RPi** 명령을 신뢰하지 않음.
  - 자체 MRM 프로파일대로:
    - 속도는 제한 감속도로 0까지,
    - 조향은 급변 억제하면서 직진/안정화,
    - 필요하면 일정 시간 후 모터 off.
  - 이건 “비상 제동/안전 정지”지, 새로운 자율주행이 아니다.

즉, **“완전 자율주행”**이 아니라 **“비상 제동·정지용 제한된 자율 제어”**라고 보는 게 맞고, 그 정도는 MCU가 충분히 할 수 있다.monolithicpower+1

---

1. RTOS in container는 실제로 어디에 쓰냐? (그리고 우리 프로젝트에 넣을지)

현실 관점에서 보면 네 직감이 맞다:

- 하드 실시간 제어(브레이크, 스티어링, 파워트레인)는 **MCU/Classic AUTOSAR/RTOS** 쪽에 몰려 있고,linkedin+2
- HPC/Linux 쪽 컨테이너 RTOS는 “핫한 기술”이긴 해도, 무조건 있어야 하는 건 아니다**.**

그래도 실제 OEM/티어1에서 RTOS in container/side-by-side를 쓰는 케이스는 있다:

- 한 하드웨어에서 Linux(Adaptive AUTOSAR 계열)와 RTOS(Classic 계열)를 같이 돌리면서:
  - RTOS: 안전 중요도가 높은 기능(예: AEB, 일부 제어 루프 등).aptiv+1
  - Linux: 인포테인먼트, HMI, 업데이트, 덜 크리티컬한 ADAS.

Aptiv 같은 곳 whitepaper에서도:

- “자동 긴급 제동(AEB) 같은 기능을 RTOS에 올리고, Linux와 나란히 같은 SoC에서 돌리는 구조”를 예시로 든다.aptiv

- 이런 경우 RTOS는 컨테이너/하이퍼바이저로 격리된 도메인으로 들어가고, Linux는 소프트 실시간/풍부한 애플리케이션 담당.

네 프로젝트에는:

- 이미 “진짜 실시간/안전 제어 = VCP-G FreeRTOS”로 가져가고 있어서,
- Jetson 쪽 RTOS in container에 자연스러운 역할이 별로 없다**.**

그래서 합리적인 선택은:

- 이번 버전:
  - RTOS in container는 과감히 뺀다.
  - 혼합 크리티컬리티는 MCU(FreeRTOS) 내에서:
    - 고우선순위: 제동/조향/속도 제어 태스크.
    - 저우선순위: 로깅/진단/통신 태스크.
      로 설명.
- 발표/보고서에서:
  - “실제 산업에서는 RTOS가 HPC SoC 안에 들어가는 케이스도 있지만,
    본 프로젝트에서는 제어 실시간성은 **MCU**로**,** 상위는 **Linux/HPC**로 분리하는 아키텍처를 채택했다”
  - 필요하면 Aptiv whitepaper, AUTOSAR Classic vs Adaptive 자료를 인용해서 “왜 이렇게 나눴는지”만 언급.linkedin+1

이러면 “핫하다”는 이유로 어설프게 끼워 넣는 느낌 없이,

- MCU 쪽은 진짜 실시간/혼합 크리티컬리티,
- Jetson 쪽은 ADAS/학습/로그/OTA/HMI
  로 역할이 깔끔하게 정리된다.

OTA 버리고 최종 데모 깊이 더하기

1. 우리가 시나리오에서 정해야 하는 핵심 포인트

지금 구조(Jetson ADAS + VCP-G MCU + MRM)를 기준으로, 반드시 수치/조건으로 명확히 정해야 할 것들은 이 정도다.

<자율주행 쪽>

- 상위 ADAS 쪽에서 내려오는 정보 정의
  - Jetson → MCU로 전달되는 값: `v_cmd`, `steer_cmd`, `mode`, `confidence`/`status flag`(룰 fallback, DL uncertainty 등).
  - "이 값들이 있어야 MCU가 상위 시스템 상태를 판단할 수 있다"는 최소 세트.
- "비정상(= failure) 상황" 정의
  - 어떤 경우를 "상위 ADAS가 더 이상 믿을 수 없는 상태(severe failure)"로 볼지:
    - 예: 안전 한계를 반복적으로 위반하는 명령,
    - 일정 시간 이상 low confidence,
    - 명령/센서 불일치, 통신 timeout 등.
      → 이걸 3~5개의 구체적인 룰(조건식)로 정해야 한다.

<MCU 쪽>

- MCU에서 하는 감시 로직 (fail-safe detection)
  - 주기(예: 10 ms), 평가 항목, 임계값:
    - `v_cmd ≤ V_MAX`, `|steer_cmd| ≤ STEER_MAX`,
    - `|v_cmd - v_meas|` 허용 범위,
    - 명령 미수신 시간 > T_timeout 등.
      → "이 조합을 만족하면 severe failure로 판정" 같은 식으로.
- MRM 시작 조건
  - 어떤 상태가 되면 즉시 **MRM** 진입,
  - 어떤 상태는 "경고 → (운전자 인계 요청/상위 소프트웨어 복구) 대기 → 안 되면 MRM"으로 할지.
    → 예:
    - 명령이 완전 이상 = 즉시 MRM.
    - confidence 낮음 = 일정 시간 대기 후 MRM.
- MRM 전략(프로파일)
  - MRM 들어간 뒤 MCU가 속도/조향을 어떻게 제어할지:
    - 최대 감속도,
    - 목표 속도(0 km/h),
    - 조향 변화율 제한,
    - 정지 후 모터 off 여부,
    - 필요하면 "짧은 직진 후 정지" 같은 룰.

<인포테인먼트>

- 시스템 반응 + 표시
  - Jetson/HU에서 보여줄 메시지: “주행 보조 제한”, “안전 정지 중”, “수동 인계 필요” 등.
  - 로그에 남길 항목: failure 종류, 트리거 조건, MRM 경과 시간 등.

이렇게 잡아야 전체 시나리오가 “그럴듯한 스토리”에서 “명확한 안전 로직”으로 바뀐다.

---

1. 이게 포트폴리오에 실제로 도움이 되냐?

도움 된다. 다만 “어떻게 써먹느냐”가 중요하다.

- 그냥 “논문 읽고 MRM 따라 했다” 수준이면 가치가 약하다.
- 하지만 네가 지금 고민하는 것처럼:
  - 산업 규정(UN R157, safe state 아키텍처)을 읽고,
  - 자기 프로젝트 구조**(Jetson + MCU)**에 맞게 문제를 정의하고,
  - 그걸 기준으로 시나리오/임계값/상태 머신을 설계했다고 정리하면,
    → 이건 “연구 논문을 읽고 시스템 설계에 반영할 수 있는 사람”이라는 시그널이다.

자동차 SW/시스템 엔지니어 포지션에서 높게 보는 건:

- 아무 API나 붙이는 게 아니라,
- 안전 요구**,** 규정**,** 아키텍처를 이해하고 시스템을 설계하는 능력이다.

지금 네가 하려는 건 정확히 그쪽이라,

- “RTOS+MCU+MRM 구조”,
- “상위 ADAS failure 감시 로직”,
- “MRM 전략 설계 및 데모”
  를 하나의 문제**/**스토리로 잘 정리하면 포트폴리오에서 꽤 강한 카드가 된다.

---

1. 이미 정립된 부분 vs 4. 아직 정립 안 된 부분

네가 써 준 분류를 조금 정리해주면:

이미 꽤 정립된 축 **(**너는 여기를 **“**잘 가져다 쓰면 됨**”)**

- 안전 아키텍처 패턴
  - fail-safe vs fail-operational, safe state 개념, degradation(기능 축소) 메커니즘.
  - 상위 도메인 컨트롤러(ADAS)를 감시하는 safety MCU 패턴.
- **RTOS MCU**로 감시·안전 모듈 구현

        - Cortex-R 계열 MCU + RTOS(FreeRTOS/Zephyr 등)로
            - 제어 태스크,
            - safety monitor 태스크,
            - watchdog, plausibility check 같은 걸 돌리는 구조.
- **MRM** 개념 자체
    - "운전자 불응 또는 시스템 심각 고장 → 안전한 감속/정지로 이동"이라는 기본 구조.
    - ALKS 규정에 나오는 정의, 감속 시간·방법에 대한 기본 틀.

여긴 너만의 새로운 이론을 만들 필요 없고, "규정/자료를 이해하고 그대로 잘 가져와 쓴다"가 목표면 된다.

아직 정립 덜 된/열려 있는 축 (여기서 네 프로젝트가 의미를 가질 수 있음)

- **"**상위 **ADAS**가 비정상**/**불신뢰 상태**"**를 어떻게 판단할지
    - 룰 기반이면: 어떤 경우를 "룰 밖"으로 볼 것인가.
    - 딥러닝이면: confidence, out-of-distribution, 이상 명령 패턴을 어떤 기준으로 위험으로 볼 것인가.
    - 카메라 기반이면: 시야불량, lane_quality 등 신뢰도 지표를 어떻게 쓰는지.
- **MCU**가 이 신호들을 조합해 **"severe failure"**로 판정하는 로직
    - threshold, timeout, 반복 횟수, 여러 신호 조합 등.
    - 이건 각 회사·연구마다 다를 수 있고, 실제로 연구·논쟁이 존재하는 영역.
- **MRM**의 구체적 전략을 어떤 수준까지 가져갈지
    - 단순 직진 정지 vs 주변 상황을 어느 정도 반영한 동작.
    - 감속 프로파일, 조향 제한, 해저드/알림 전략.

여기는 너 프로젝트가 **"**산업적 문제를 축소해서 실험하는 예시**"**가 될 수 있는 부분이다.

요약하면:

1. 우리가 시나리오에서 해야 할 일은
    - "어떤 상태를 상위 ADAS failure로 볼지"와
    - "그때 MCU가 어떤 룰로 MRM을 시작·수행할지"를 숫자와 조건으로 명확히 정하는 것.
2. 이 작업은
    - 단순 기술 나열이 아니라,
    - 규정/논문을 읽고 시스템 설계에 반영하는 훈련이라
      → 조천식/자동차 SW 포트폴리오에 분명 도움이 된다.
3. 이미 있는 개념(안전 아키텍처, RTOS MCU, MRM) 위에,
    - 아직 열려 있는 부분(ADAS failure 판정, MRM 세부 전략)을
    - "학생 프로젝트 스케일로" 풀어내겠다는 포지션이면
      → 너무 크지도, 너무 뻔하지도 않은 적당한 난이도다.

어떤 자료를 근거로 시나리오를 만들까?

1. 무엇을 깊게 팔지 (추천 세트)

네 시나리오 구조 기준으로:

- 1순위: **UN R157 (ALKS** 규정**, MRM** 포함**)**globalautoregs+1
  - 이유:
    - “ADAS failure / severe failure / MRM / safe stop”을 공식 용어와 구조로 정의해줌.
    - “언제 시스템이 더 못 달리고, 어떻게 최소 위험 상태로 가야 하는지”에 대한 기본 프레임.
  - 너가 여기서 뽑을 것:
    - failure/ODD/MRM 개념,
    - 속도·시간 관련 기본 사상,
    - “failure → 인계 요청 → MRM” 구조.
- 2순위: **Safe state/Fail-operational** 아키텍처 자료 **(Elektrobit** 글**/**논문**)**elektrobit+1
  - 이유:
    - “메인 시스템 + 모니터 시스템 + degrade + safe state” 아키텍처 패턴을 정리.
    - VCP-G를 Safety MCU로 두는 근거, 감지/제한/정지의 단계적 전략을 잡을 수 있음.
  - 여기서 뽑을 것:
    - safe state 정의,
    - 단계적 degradation(속도 제한, 기능 축소, 정지),
    - 모니터 ECU(=VCP-G) 역할.

이 두 개만 깊게 파도:

- Jetson ADAS failure 정의
- MCU 감시/Fail-safe/MRM 조건
- HU/HMI 메시지·로그

까지 모두 이 두 소스에서 뼈대를 가져올 수 있다**.**

추가로 시간 되면:

- Infineon “Safety MCU in ADAS” 자료에서 Safety MCU 역할 요약 1–2줄만 가져오면 된다.infineon+1

---

2. 어떻게 사용할지

- 표준/논문을 “이론 백과사전”처럼 다 외우려 하지 말고,
  - 각 문서에서 우리 설계에 직접 쓰일 문장**/**그림**/**개념 몇 개만 뽑는다**.**

예:

- R157에서:
  - ALKS failure/severe failure 정의,
  - MRM의 목적·트리거,
  - 운전자 인계 + MRM 흐름 그림/문장.unece+1
- safe state/Fail-operational 논문에서:
  - degrade 메커니즘(속도 제한, 기능 off, safe stop),
  - 메인–모니터 아키텍처,
  - safe state를 어떻게 설계하는지 예시.elektrobit+1

→ 그걸 근거로:

- "그래서 우리 프로젝트에서는 failure를 이렇게 정의했다"
- "그래서 우리 Safety MCU(VCP-G)는 이런 역할을 한다"
- "그래서 MRM/Fail-safe 전략을 이렇게 설계했다"

라고 연결시키면 된다.

---

3. 왜 이 방식이 좋은지

- 현실성:
  - 학부/석사 지원 수준에서 모든 표준/논문을 넓게 아는 것보다,
  - 핵심 **1–2**개를 설계에 직접 녹여서 설명하는 게 훨씬 강한 인상을 준다**.**
- 평가자 관점:
  - "R157과 safe state 아키텍처를 근거로, 자기 프로젝트 구조·조건·수치를 설계했다"는 건
  - **"**논문**/**표준을 실제 시스템 설계에 적용할 줄 아는 사람**"**이라는 시그널이다.

그래서 내 의견은:

- **R157 + safe state/Fail-operational** 글**(**또는 논문**) 1**편
  이 두 개를 메인으로 깊게 파고,
- 나머지는 필요할 때 참고만 하는 정도로 두는 전략이 가장 좋다.

IdeaV2. DCAS

4/1까지 고민 내용 정리

1. 아키텍처: 젯슨 - 라즈베리 - mcu - 차
2. 차량 제어 로직을 MCU로 옮김: 라즈베리파이의 효용성?
3. MCU의 역할 정의
1) Zonal Controller 자체의 메인 MCU: 각 구역(Zone)의 전력 분배와 센서 데이터의 실시간 라우팅, 말단 부품(액추에이터) 제어를 안전하게 수행합니다.

2) Central Compute 내의 Safety Co-processor: 중앙 컴퓨터에 탑재된 무거운 고성능 SoC(예: 젯슨, 텔레칩스 D3 등) 바로 옆에 붙어서, 메인 SoC가 뻗지 않고 잘 살아있는지 감시하는 역할을 합니다. 문서의 기능 블록도는 물리적인 상자의 모양만 달라질 뿐, 시스템 내부의 논리적 아키텍처로는 그대로 적용됩니다.

감시 역할을 위한 기술 요소

1) 하트비트(Heartbeat) 모니터링 체계 구축: * 메인 SoC의 자율주행 소프트웨어는 10ms 단위처럼 일정한 주기마다 Safety MCU로 "나 정상적으로 살아있어"라는 하트비트 신호(또는 Watchdog 토글 신호)를 보내야 합니다.

Safety MCU는 이 신호가 정해진 시간 내에 오지 않거나, 비정상적인 데이터가 수신되면 즉시 **'메인 시스템 고장'**으로 판단합니다.

2) Lockstep (록스텝)(컴구) 구조 이해: * Safety MCU 내부는 보통 두 개의 코어가 똑같은 연산을 동시에 수행하고, 그 결과값을 실시간으로 비교(Lockstep)합니다. 우주선이나 항공기에 들어가는 기술로, MCU 자체에 물리적인 오류(예: 방사선으로 인한 비트 플립)가 생겨도 즉각 알아챕니다.

3) MRM (Minimal Risk Maneuver, 최소 위험 운행) 제어권 전환:

메인 SoC가 고장 났다고 판단된 순간, 조향(스티어링)과 제동(브레이크)의 제어권은 즉시 Safety MCU로 넘어와야 합니다.

Safety MCU는 복잡한 자율주행 연산은 못하지만, 비상등을 켜고 차선을 유지하며 차량을 갓길에 안전하게 세우는 **비상 정지 로직(MRM)**을 미리 가지고 있어야 합니다.

5. 공식 근거 자료
1) UNECE R157: 레벨3 ALKS를 양산하려는 OEM들은 UNECE R157을 기준으로 failure·MRM·운전자 인계 로직을 설계해야 한다.
UNECE R157란?
UNECE 산하 자동차 규제 조화 포럼(WP.29)의 네 가지 주요 규제안 중 하나. 규정 157호.
대상 기능: 고속도로·유사 도로에서 차선을 유지하며 자동으로 주행하는 ALKS (레벨3).
이 새로운 지침에 따라 레벨 3 차량은 시스템이 차량 기능을 관리하는 동안 운전자가 도로에서 눈을 떼도 되도록 허용합니다. 그러나 운전자는 언제든지 개입할 수 있도록 항상

대기해야 하며, 이를 통해 인간의 감독을 완전히 배제하지 않으면서도 시스템이 안전하게 작동할 수 있도록 해야함.
이게 어떤 내용을 담고 있나:
https://auto2xtech.com/regulation-level-3-autonomous-driving-2021/
산업에서 어떻게 받아들이나: https://kidd.co.kr/news/242973
2) 일레트릭비트:
https://www.elektrobit.com/blog/safe-state-architectures-degradation-mechanisms-failures/

6. 최종 시나리오
1) 문제: 카메라 기반 LKAS가 한계를 만나는 상황. R157은 LKAS의 문제 상황에 대한 규제를 담고 있음. 자율주행 팀의 LKAS는 완성된 상태. 따라서 LKAS에서 카메라 한계에 대한 문제를 다루는게 좋을 듯.
2) 구체적 문제: 카메라 기반 LKAS가 안되는 여러 상황을 주고, 각 상황의 위험도에 따라 수동 운전/MRM 등을 구현.
3) 큰 틀: UN R157(ALKS)의 transition demand 및 Minimum Risk Manoeuvre 개념
ODD 벗어남/ALKS failure 징후 → transition demand
운전자 미응답 → 일정 시간 후 MRM 개시
MRM은 위험 최소화·정지 지향.
4) 세부 내용:
상황을 수치로 어떻게 변환할지
임계값을 얼마로 둘지
MRM 로직의 상세한 상태 기계/감속 프로파일을 어떻게 설계할지

7. 오토사, iso는 뭐지?
1) 오토사:
2) iso26262: 전자제어시스템의 오작동 위험을 방지

8. 교수님께 질문: 규제나 표준이 너무 많아서 어떤 것을 참고해서 우리 시스템에 적용하는게 좋을 지 모르겠음.

4/2 미팅 이후 업데이트

교수님 말씀

- **<u>R157 << DCAS R171</u>**

1. R157 아디어에서는 제어권 전환이 중요: 운전석에 스티어링 휠가 운전자가 휴먼 인더 루프로 돌아가야 함. 그냥 가다가 멈추는 것은 데모에서 보여줄 게 없음. 조이스틱으로 수동 운전은 LKAS safety regulation.
2. 현재 아이디어는 새로움이 없음. 컨벤셔널함. 제어권 전환은 5,6년 전에 핫했음. 지금은 중요하지만, 핫하지 않음. (요즘 다 만나서 하는 얘기는 LLM, VLM.)
3. **LV2+**에서 큰 관심사는 디카스 **R171. DMS. In cabin driving monitor.** 요즘 **FSD**를 감독형이라고 함**.** 사람이 감독한다는 전제에**.** 이 경우에는 핸즈온은 안하더라도 눈만 감시하는 것 가능**.** 요즘은 **LKAS safety**보단**,** 운전자가 주의 집중을 하고 있지 않을 때**.** 요즘 자동차회사가 다 이거 고민 중임**.**
4. 학생 프로젝트는 향후 **10**년 뒤의 주제에 대한 것을 다루는게 매력적**. -->** 왜지**?**
5. + rtos in container를 해봤는데 스케줄링이 잘 안돼있음.

- **<u>VLM</u>**

1. **In Cabin**에서도 비전기반**,** 포즈넷도 해야겠지만**,** 랭귀지 모델로 모니터링하는 것도 **safety**에 최대 관심사**(+**디카스**). 7,8**년 후에 집중하는 사람들에게 핫함**.**
2. 컴퓨팅 파워 많이 필요

- **<u>NPU</u>**

1. 전통적인 2016 엔드 투 엔드 모델은 텔레칩스 NPU에서 잘 돌아감.
2. **VCP**는 그냥 리얼 타임**.** 그것 보단 **NPU**를 사용하는게 더 매력적**. GPU**는 많이 쓴느데 흔히 쓰지 않음**.**
3. 열심히 하는 것 보단 유니크한 것**. NPU**를 쓰는게 더 유니크할 듯**.**
4. 지원자 100 중 99는 파이토치로 LKAS.

- **<u>In cabin monitoring:</u>** <u>미니멀하게는 **usb**카메라로**.**</u>

1. 핸즈 온 리퀘스트는 인스트룸ㅓㄴ트 클러스터 **GUI**로 보내는 것**.**
2. 핸즈온도 카메라로 감지 가능**.** 지금 방식은 스트어링 휠에 힘을 주는 것을 감지**,** 정전식**.** 앞으로는 다 카메라로**.**
3. 잡고 있는지 등은 **vlm**이 기가마ㄱ히게 해줄 것**.**
4. 아이즈 온**,** 핸즈 온 안하면 **MRM(RMF)**

상황에 맞춰서 현실적으로 조정.

교수님 말씀에 대한 내 생각 및 리서치

<내 생각>
메인: R171에 근거한 카메라, VLM 기반 In cabin monitor. 이에 대한 대응을 하려면 여전히 VCP필요.(+ 완성된 모델 NPU)

- 요구사항 정립

1. 임베디드 + ADAS인 의미가 있어야함
2. 컨벤셜<<미래 지향적
3. 챌린징
4. 2달 완성
5. 데모에서 시각적으로 보여줄 것이 있어야함.
6. 이전 기수보다 AI 툴 발전. 더 잘해야함. 나는 이전 기수보다 더 잘했다는 소리를 듣고 싶음.

문제 상황이 LKAS에서 DCAS로 변경, 상황 수치화/정량화를 VLM이 담당, 운전자가 다른 활동을 과하게 못하는게 핵심. 문제 터지고 대응이 핵심인 R157과는 다름. 즉, Fail - safe 로직보단 예방이 포인트.
한계: VLM이 리소스 많이 잡아먹음. 현재 우리가 얼마나 사용할지 모름.

<질문>

- 그냥 제어권 전환하는 것은 전통적인데, in cabin monitor는 뭐가 다른거지? 문제 상황만 다른거 아닌가?
  - **R157 쪽은 "차가 한계를 만났을 때" R171+DMS 쪽은 "사람이 한계를 벗어났을 때"**
- 디카스란?
  - DCAS = Driver Control Assistance Systems.
  - 정의 (UN R171): "차량의 종·횡방향 제어를 지속적으로 도와주지만, 운전 과제를 완전히 대신하지는 않는 시스템. 즉, SAE Level 2, 운전자가 항상 책임을 지고 감독해야 하는 보조 운전 시스템이다.

- R171이란?
  - UN Regulation No. 171 = Driver Control Assistance Systems에 대한 승인 규정.
  - 2024년 9월 발효, DCAS의 안전·성능 요구사항을 전 세계적으로 통일하는 규제.
  - 목표: DCAS 사용 시 발생할 수 있는 "오용·남용(misuse/abuse)"과 과신(overreliance)를 줄이는 것. 드라이버가 운전 외 다른 활동을 과하게 하지 못하게 설계하라고 규정

- vlm이란?
  이미지/영상(vision) + 텍스트(language)를 같이 다루는 멀티모달 모델.
  예: 사진 보여주고 텍스트로 설명하거나, "이 운전자는 핸들을 잡고 있냐?" 같은 질문을 이해해서 답하는 모델.
  카메라로 본 실내 영상 + 텍스트 명령/질문을 함께 처리해서 자연어 수준으로 이해·판단하려는 시도

<후속 질문>

디카스는 Lv.2+인데 어떻게 이게 더 미래지향적인 거지? 실제로 테슬라 FSD를 감독형이라고 하면서 핫해?
R157은 레벨3(운전자 손·눈 떼도 되는 조건부 자동운전)이라 “완전 자동화”에 가까운 규제다.
그런데 현실 시장에서 당장 대량으로 깔리는 건 레벨2+/DCAS: 테슬라 FSD Supervised, GM SuperCruise, 현대 HDA2 같은 “운전자가 감독하는 고급 보조운전”이다.
UN R171은 이 DCAS용 통일 규제다. “운전자를 계속 깨어 있게, 책임 있게 유지시키는 메커니즘”을 요구해서, 테슬라 FSD 같은 감독형 시스템이 딱 이 범주에 들어간다.
→ 그래서 산업이 “향후 10년 동안 가장 많이 팔릴 현실적인 자동화”로 보고, R171/DCAS+DMS를 핫한 주제로 보고 있는 거다.
vlm을 자연어로 처리해서 어떻게 활용? 이걸 safety와 연결이 돼야하잖아.
상위 SoC에서 영상+텍스트를 보고 이미 “핸들을 잡고 있음/안 잡고 있음, 눈은 도로를 보고 있음/아님”을 의미 단위로 정리해서 넘겨줌. 우리는 그걸 true/false나 OK/WARN/CRIT 같은 간단한 상태로만 받아서 안전 로직에 넣으면 됨
즉, “복잡한 비전 규칙 설계·수치화”는 VLM/AI 쪽에 맡기고, 임베디드는 그 출력에 대해 언제 경고/제한/감속·정지할지 상태 머신과 임계값만 설계하면 되는 구조
'R157 쪽은 “차가 한계를 만났을 때” R171+DMS 쪽은 “사람이 한계를 벗어났을 때”'라고 했는데 그럼 MRM때와 달리 MCU또는 임ㅂㅔ디디드에서 하는 일이 달라지나?
근데 DCAS제한, 경고, 감속, 정지는 젯슨에서도 충분히 감독할 수 있잖아. 꼭 감독 주체가 MCU일 필요가 있나?
fail-safe 내용 구현 정도
'ISO 26262에서는 높은 ASIL 기능(감속·정지, steering cut-off 같은 safety goal)을 단일 복잡 SW/SoC에만 의존하지 말고, 독립적인 진단/감시 채널을 요구하는 패턴이 많다.' 에 근거해서 MCU가 간략한 감시.
→ “젯슨이 미쳐 날뛰거나 죽어도, MCU가 최소한의 브레이크 제어/토크 컷을 보장한다”는 구조를 위해 MCU를 둔다.
R171이 무슨 내용인지?
핵심 요구사항 (임베디드 관점에서 중요한 것만):
운전자 참여·감시 요구: “DCAS는 운전자의 continuous involvement를 평가할 수 있는 수단을 가져야 한다.”
모터리(motoric) – 핸즈온(스티어링 입력)
비주얼(visual) – 시선/머리 자세(gaze direction / head posture).
Engagement 모니터링: “DCAS는 핸즈온 또는 eyes-on road 또는 둘 다를 통해 운전자 engagement를 모니터링하고, 부족 시 경고를 줘야 한다.”
경고에도 불구하고 참여 회복 안 되면 시스템을 해제하거나 안전한 상태로 전환하는 failsafe 응답 필요.

- DCAS, R171이 왜 핫하지?

  상용화됐지만, 규제 강화와 L2+ 혁신으로 핫합니다.

  - R157 vs R171: R157은 기본 ADAS(LKAS 등) 제어권 전환 중심. R171은 DCAS(L2+)로 DMS 필수 – 운전자 감독/집중 모니터링 강조. **FSD**처럼 **"**감독형**"** 자율주행에서 핸즈오프 가능하지만, 아이즈온**/DMS**로 오용 방지**.**
  - 왜 핫?
    - 2024~2025 규제 시행: OEM(현대 등)이 DMS 고도화 중. 카메라 기반 핸즈온(정전식 → 비전), MRM 필수.

    - 미래 10년 주제: **L3+**로 가며 "인캐빈 모니터링"이 핵심. **VLM**으로 "왜 산만한지" 설명 가능 – 기존 센서보다 똑똑.
    - 상용화됐지만, VLM/NPU 통합이 차별화. 교수님처럼 "유니크"하게 NPU 포즈넷 + VLM.

- 정전식도 충분한 것 같은데 굳이 카메라?

  객관적으로 보면, 정전식이 "나쁘다"기보다, 카메라가 해결하는 문제가 더 많아서 핫한 것입니다. 그리고 이미 상용화된 기능이 있어도, 규제 대응·오탐 감소·상황 인지 범위 확장 때문에 여전히 연구/산업 가치가 있습니다.

  <u>왜 카메라인가</u>

  정전식은 핸들을 잡았는지 같은 아주 특정한 신호에는 강합니다. 하지만 손이 핸들에 닿아 있지만 주의는 안 하는 경우, 시선이 전방을 보는지, 졸음/휴대폰/다른 행위 같은 건 정전식만으로는 알기 어렵습니다.
  즉, 정전식은 "접촉"을 보지만, 카메라는 행동과 맥락을 봅니다.wikipedia+1

  <u>왜 **DKAS/DMS**가 계속 뜨나</u>

  이 분야가 핫한 이유는 "새로운 센서 하나" 때문이 아니라, 규제와 제품 요구사항이 바뀌었기 때문입니다. R171/DCAS처럼 L2+에서 운전자 감시가 중요해지면서, 단순히 핸들을 잡았는지보다 운전자가 실제로 주의를 두고 있는지가 더 중요해졌습니다.appliedintuition+1

  <u>이미 구현된 기술로 충분한가</u>

  부분적으로는 맞습니다.

    - 정전식: 핸즈온 검출에는 충분히 유용합니다.
    - 카메라+전통 **CV**: 아이즈온, 머리 방향, 자세 추정에 이미 많이 쓰입니다.
    - **VLM**: 설명 가능성, 복합 상황 해석에서 추가 가치가 있습니다.google+1

  하지만 "충분하다"와 "끝났다"는 다릅니다. 업계는 지금도 오탐을 줄이고, 다양한 운전자 자세/가림/조명/안경/마스크 상황을 버티는 것, 그리고 더 적은 센서로 더 많은 의미를 추론하는 것을 원합니다. 그래서 기존 기술 위에 카메라/포즈/비전-언어 모델을 얹는 방향이 계속 연구됩니다.42morrow.tistory+1

  <u>객관적인 판단</u>

  정리하면:

    - 정전식만으로도 가능한 범위가 있습니다.
    - 하지만 **DMS/DKAS**가 핫한 이유는 "핸들 접촉"을 넘어서 "운전자 상태 전체"를 봐야 하기 때문입니다.

- 그래서 카메라는 더 비싸고 복잡해도, 감시 범위가 넓고 규제 대응력이 높아 가치가 있습니다.appliedintuition+1

교수님 말씀의 핵심

교수님이 말한 포인트는 "기술이 없어서 핫하다"가 아니라,
"이미 있는 기술을 넘어서, 규제·안전·미래 제품 관점에서 더 의미 있는 문제를 풀어야 한다"는 뜻에 가깝습니다.
즉, 학생 프로젝트로는 정전식 단독 구현보다 카메라 기반 **DMS** + 필요시 정전식 보조가 더 연구/데모 가치가 큽니다.naver+1

원하시면 다음 답변에서
"정전식 **vs** 카메라 **vs** 하이브리드"를 **R171** 관점으로 아주 단순한 표로 정리해드릴게요.

VLM - 영교

- 국민대 서버에 이미 Qwen-2B 모델이 있음. 이거 써야함.
- 젯슨에서 카메라 입력 받고 인코딩+시스템프롬프트를 국민대 서버에 전송. → **API**사용
  - 이 때, 시스템프롬프트에 어떤 형식(json)으로 답변을 받을지 보내줘야함.
  - 근데 멍청해서 제대로 안 줄때도 있음. 학습시켜야함.
  - 인코딩: base64
- 다른 VLM모델 쓰고 싶으면, 랩실 컴퓨터에 meta의 Llama 설치해서 사용.
  - 무료임
  - 서버가 한국이 아니라서 빠를 듯.

- 전윤성님
  영교님의 API key는 아래와 같습니다.
  api_key :
  81cbe888efea8c89da139c5cc8194393c1ead203e11e85a9a5a721428c5a2517

  사용방법은 총 3가지입니다.
  1. 터미널에서 curl 명령어
     - 터미널에서 1회성으로 보낼 수 있는 방법입니다.

  2. open-webui 기반 사용법
     -GPT처럼 웹 기반으로 사용할 수 있는 방법입니다.

  3. python 기반 사용법
     - python을 이용한 사용 방법입니다.

  개발에 사용하실 예정이시라면 3번 방법이 적합해 보이긴 합니다.

간단하게 설명은 첨부한 "vLLM 외부 접속 매뉴얼.txt"를 참고하시면 됩니다.
참고로 현재 서버에 로딩된 모델은 Qwen3-VL-2B-Instruct 모델입니다.

마지막으로 영교님이 평소에 개발하시는 시간대를 한국시간기준으로 알려주시길 바랍니다.
연구실에서 **vLLM**을 잠깐 내릴 수도 있고 사용중일 수도 있기 때문에 서로 겹치지 않게 하기 위함입니다. → 한국 시간 기준 **19**시 **- 24**시

Qwen b2가 잘 잡는 맥락

- 핵심만 말하면, Qwen b2급 VLM은 "행동 카테고리"를 나누는 데는 꽤 쓸 만하고, 세밀한 시선·자세 정량은 약하다고 보는 게 안전하다.

- 잘 잡는 쪽 (맥락 태그용으로 쓰기 좋은 것)

  이런 건 비교적 안정적으로 구분할 수 있다.

  - 휴대폰 관련
    - 손에 폰 들고 있음, 화면을 보고 있음, 통화 중(귀에 대기, 스티어링에서 손 떨어짐)
    - "폰 사용 중 vs 운전 집중" 같은 coarse 클래스는 잘 나옴.
  - 물건/행동 카테고리
    - 먹는 중(음식, 음료 캔/컵 들고 있음)
    - 흡연(담배, 연기, 라이터)
    - 실내 대화(옆 사람 쳐다보고 말하는 자세)
  - 자세·시선의 "영역 단위"
    - 전방/센터스택/센터콘솔/동승자 쪽을 쳐다보는 정도의 영역 구분.
    - 즉 "앞을 안 보고 옆을 본다" 수준의 판별은 비교적 괜찮음.

  이 레벨이면, 네가 원하는:

  - `vlm_reason = phone / eating / talking / smoking / away` 같은 맥락 라벨을 뽑아서
  - "폰이면 브레이크 펄스, 졸음이면 감속, 자리 비우면 즉시 제한" 같은 정책에 쓰기에는 충분하다.

- 약한 쪽 (여기까지 믿으면 위험한 것)
  - 정밀 시선 추정
    - 눈동자 각도 몇 도, 차로선까지 포함한 "정말 도로를 보고 있는지" 수준은 전용 gaze 모델이 필요하고, VLM에 다 맡기기엔 부정확하다.
  - 미세한 졸음/피로 신호
    - 반쯤 감긴 눈, 깜빡임 주파수 같은 건 DMS 특화 모델보다 떨어진다.
    - "눈 감고 있음" 정도의 극단 케이스는 잡아도, 실질 DDAW 수준은 어렵다.
  - 타이밍·프레임 단위 판단

- VLM은 한두 프레임 기준의 인식은 가능하지만, “최근 10초 동안 몇 번이나 폰을 봤는지” 같은 시계열 분석은 별도 로직이 필요하다.

미디어파이프 포즈넷

MediaPipe Pose는 Google AI Edge에서 개발한 실시간 인간 자세 추정 프레임워크로, 모바일/엣지 디바이스에서 33개 3D 키포인트를 60fps 이상 처리합니다. 2019년 MediaPipe 프로젝트의 일부로 시작해 BlazePose GHUM 모델을 기반으로 진화했습니다.puleugo+1

배경

문제 해결: 전통 컴퓨터 비전에서 자세 추정은 GPU 의존적이고 느렸습니다. MediaPipe는 모바일 우선으로 설계되어 스마트폰, Jetson, NPU에서 실시간 동작합니다. DMS/피트니스/AR/VR 등 엣지 AI 응용에 최적화.google

진화 과정:

text
```
TensorFlow Lite (2017) → BlazePose (2020) → MediaPipe Pose (2021+)
├── 포즈넷 (웹 17 keypoints)
└── MediaPipe (엣지 33 3D keypoints)
```

원리

3단계 파이프라인으로 작동합니다:

1. 검출**(Detector)**: BlazePose TH 모델 – 사람 상반신 후보 영역 탐지 (YOLO-like).
2. 키포인트 예측**(Keypoint Model)**: BlazePose GHUM – 33개 3D 랜드마크 회귀. **GHUM(Google Human Motion)**으로 관절 각도/깊이 추정.puleugo
3. 트래킹**(Tracker)**: Kalman 필터 – 프레임 간 일관성 유지 (깜빡임 방지).

수학적 핵심:

text
```
입력: RGB 이미지 (224x224)
중간: Heatmap (각 키포인트 위치 확률)
출력: 33 × (x,y,z,visibility)
     x,y ∈ [0,1] 정규화 좌표
     z: 상대 깊이 (엉덩이 중심 기준)
     visibility: [0,1] 가려짐 확률
```

세계 좌표 변환: 카메라 내부 파라미터로 2D → 3D 실세계 미터 좌표.puleugo

기술 구조

모듈화 파이프라인:

text

```
카메라 프레임 → 전처리(정규화)
              → Detector (상반신 박스)
              → Keypoint Model (33 landmarks)
              → Post-processing (smooth, 3D)
              → 출력 (랜드마크 + 연결선)
```

최적화:

- 모델 복잡도: 0(라이트)~2(헤비) – 속도/정확도 trade-off.
- 배경 분할: Segmentation mask로 사람/배경 분리.
- **Holistic** 모드: Pose + Face + Hands 동시 처리.

DMS에서의 강점

R171 규정 핵심 요구사항 충족:

- 핸즈온: 손목-어깨 벡터 → 핸들 위치 판단.
- 아이즈온: 코-눈 각도 → 시선 방향.
- 실시간 **+** 저전력: NPU/Jetson에서 30-60fps, VLM(1-2s) 보완.google

한계: 복잡 포즈(가려짐, 여러 사람)에서 visibility↓ – VLM과 조합 필수.

MediaPipe는 엣지 **AI**의 표준으로 자리 잡았습니다. 자세한 파이프라인 다이어그램이나 논문 원문 필요하시면 더 찾아드릴게요.google+1

R171

- 네가지 핵심

1. 기능 성능 (Functional Performance)
   - AEBS, LKA, LDW 필수 탑재

2. 동적 제어 (Dynamic Control)
   - 차량이 예측·제어 가능 상태 유지

3. 시스템 한계 (System Limits)
   - ODD 이탈/고장 시 안전 조치

4. 인간-기계 인터페이스 (HMI) ← 핵심
   - 운전자 감독/경고/재참여 요구

- 그 중 핵심은 HMI

1) Hands-On Request (HOR)

- 핸들에서 손 떼고 3초 → 경고

- 5초 이상 → 강제 HOR 발령

2) Eyes-On Request (EOR)

- 속도 50km/h↑: 3.5~5초 시각 이탈 → 재참여 요구

- 속도 10~50km/h: 6초 이상 시각 이탈 → 재참여 요구

3) Direct Control Alert (DCA)

- EOR 무시 → 5초 내 DCA (시각+음향/촉각)

- 계속 무시 → 10초 내 "운전자 불응 절차" 시작

4) Disengagement Detection

- 30분 이상 개입 없음 → 시스템 30분 비활성화

DCAS 대응 방법 조사

운전자를 “다시 집중시키는” 방법은, 연구 기준으로 보면 크게 세 축이 있어. 지금 네 로직은 그 중 일부를 이미 쓰고 있고, 조금만 더 의식적으로 가져오면 된다.

---

1. 경고 모달리티: 시각 + 음성 + 하프틱

연구/프로토콜에서 공통으로 나오는 건 멀티모달 경고야.

- 시각 + 음성 조합
  - 시각(HUD/클러스터)은 “무슨 일이, 왜 일어나는지”를 설명하는 데 좋고,tandfonline+1
  - 음성/비프는 반응 시간을 줄이는 데 효과적이라고 많이 나온다.sciencedirect+1
  - 너가 쓴 “무슨 일이 일어나는지 설명해주는 HMI” 문구는 이쪽 연구랑 딱 맞는다.fis.tum
- 하프틱(브레이크 펄스, 스티어링/시트 진동)
  - 교차로 위반 경고 연구에서, 짧은 브레이크 펄스(저크, 지속, 펄스 횟수 설계)가
    - 그냥 시각/비프보다 훨씬 높은 개입률을 보였다는 결과가 있다.vtechworks.lib.vt
  - 또 다른 연구에서도, 시각+음성+하프틱 조합이 **take-over** 요청(**TOR**)에서 가장 좋은 성능을 보였다고 한다.d-nb

→ 너 로직에선:

- `DRIVER_WARNING`에서
  - 클러스터 문구 + 짧은 음성/비프 + 약한 하프틱(스티어링·시트·브레이크 펄스)을 조합해서 쓰는 게, “집중 환기” 측면에서는 가장 근거가 있다.vtechworks.lib.vt+1

---

2. 메시지 내용: “무슨 상황 + 지금 뭘 하라”

단순 비프보다 구체적인 음성 메시지가 반응을 더 잘 끌어낸다는 연구가 있다.sciencedirect

- “삐-삐-삐”보다
  - “앞을 보세요”, “핸들을 잡으세요” 식의 행동 지시형 문장이
  - 지각-반응 시간을 줄이고, 실제 행동율을 올린다는 결과.medien.ifi.lmu+1
- MRM 연구에서도,
  - “지금 차가 스스로 갓길 정지 중이다 / 핸들을 잡으면 직접 제어 가능하다”처럼
  - 상황+추천 행동을 같이 보여주는 HMI가
    - 운전자 이해도와 개입 품질을 높였다고 보고된다.fis.tum

→ 네 로직에 적용하면:

- `DRIVER_WARNING`

    - “전방을 주시하세요. 핸들을 잡으면 경고가 해제됩니다.”
- DRIVER_UNRESPONSIVE
    - “운전자 미응답, 안전을 위해 감속합니다. 핸들을 잡으면 제어를 다시 할 수 있습니다.”

처럼 상황 **+** 지금 해야 할 행동 두 가지를 명시적으로 항상 넣는 게 근거 있는 설계다.sciencedirect+1

---

3. 개입 패턴**:** 부드러운 감속 **+** 지속 자극

집중을 유도하는 방법으로 차량 거동을 이용한 부드러운 감속·진동도 연구가 된다.

- MRM 쪽 연구
    - 단번에 큰 감속/급조향보다, 완만한 감속 **+** 명확한 **HMI** 안내가
        - 사고 위험을 줄이면서도 운전자 개입을 잘 유도한다고 나온다.ko-haf+1
- 좌석 진동/리듬 연구
    - 장기 자율 주행 환경에서, 저주파 좌석 진동이 각성도와 자세 안정성 유지에 도움이 될 수 있다는 결과도 있다.pmc.ncbi.nlm.nih
    - 이건 “졸음 운전 예방” 쪽 supportive 전략.

→ 네 구조에서 쓸 수 있는 포인트:

- DRIVER_WARNING
    - 속도/도로 상황 괜찮을 때는 매우 완만한 감속 시작 **+** 짧은 하프틱으로 “지금 뭔가 일어나고 있다”는 감각을 줌.
- DRIVER_UNRESPONSIVE
    - 감속 기울기↑, 경고 강도↑, MRM 준비
    - 단, “차를 일부러 세게 흔드는” 급 jerk는 피하고, 연구에서 제안되는 수준의 짧은 펄스/진동만 사용.wiki.unece+1

---

4. 네 로직에 바로 붙일 수 있는 **“**집중 유도 아이디어**”**

위 연구를 네 현재 설계에 맞게 번역하면:

- 모달리티
    - WARNING: 시각 + 음성(문장) + 약한 하프틱
    - UNRESPONSIVE: 시각 + 음성(더 긴 문장) + 조금 더 강한 하프틱 + 감속
- 메시지
    - 항상 “지금 상태 설명 + 운전자에게 요구되는 행동” 두 부분을 포함.
    - 예:
        - WARNING: “앞을 보지 않고 있습니다. 핸들을 잡고 전방을 주시하세요.”
        - UNRESPONSIVE: “운전자 미응답, 차량이 자동 감속 중입니다. 핸들을 잡으면 직접 운전할 수 있습니다.”
- 패턴
    - 폰 사용: 짧은 브레이크 펄스 여러 번 + “전방을 보세요”

- 졸음: 연속 경고음 + 완만한 감속 + “가까운 휴게소로 안내합니다.”
  → 이건 VLM 맥락 얹어서 구현하면 네 차별화 포인트가 된다.seeingmachines+1

이 정도가, 논문/연구에서 근거가 있는 “운전자 집중을 다시 끌어오는 방법”들이다.
지금 네 설계 방향(설명형 HMI + 감속 + 하프틱)을 그대로 쓰되, 모달리티 조합과 메시지 구조를 의식적으로 설계했다고 포트폴리오에 써주면 좋다.

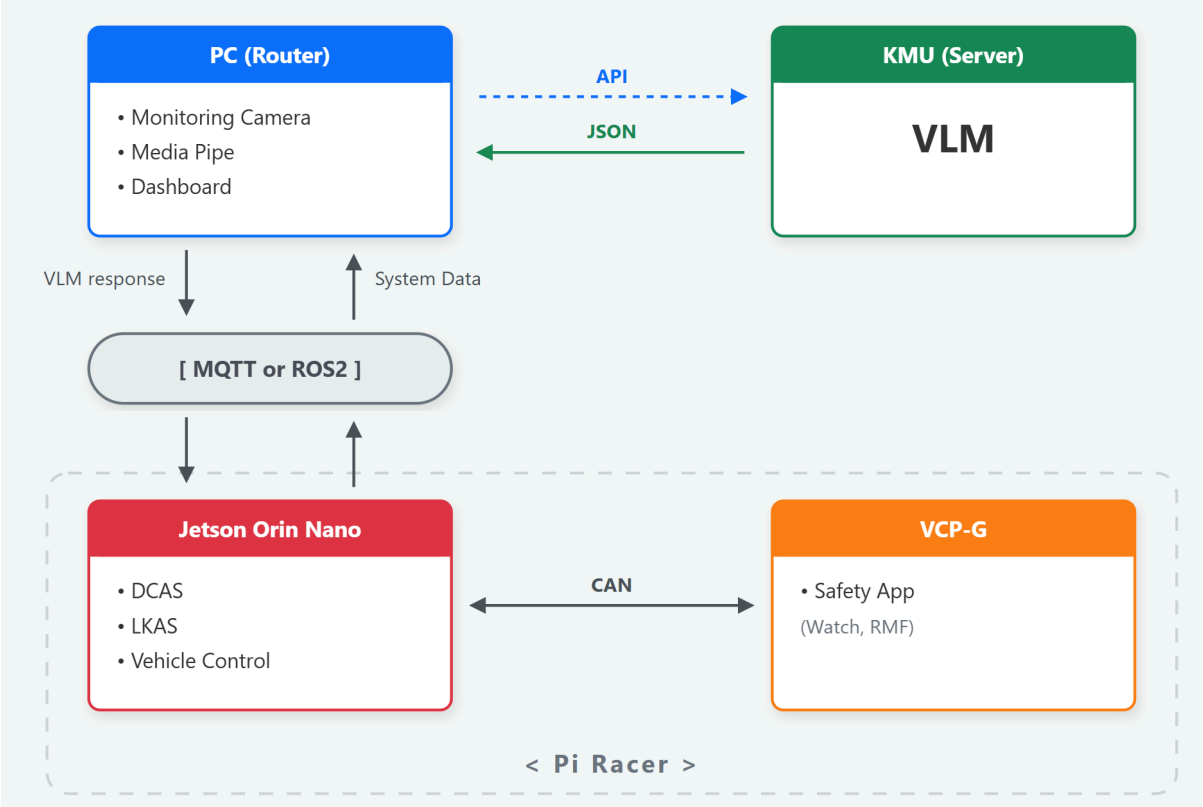
현재 결론

1. 주제: DCAS
2. 문제 정의: 차가 아니라 운전자가 한계를 벗어났을 때, DCAS가 어떻게 경고·제한·감속할 것인가 (R171 driver engagement, misuse/overreliance 관점).
3. 목표: R171 맥락에서 운전자 상태 전체(행동·맥락)를 감시하는 카메라 기반 DCAS 프로토타입 구현
4. 핵심 요소
    a. VLM: 2차 분석(맥락 분석)
    b. 미디어 파이프 포즈(NPU최적): 1차 분석()
    c. 상태 대시보드 GUI: 운전자 상태, 알림, 시스템 상태, …
    d. Jetson Orin Nano: 차량제어, DCAS모듈(LKAS포함)
    e. VCP-G: jetson heartbeat 감시, 주행 불가 시, MRM 보장 → 후순위 구현
5. 최종 데모 시나리오
    a. 정상 DCAS on, 운전자 집중 (hands-on, eyes-on) → LKAS 유지.
    b. 운전자 핸들 손 떼거나, 시선 크게 이탈 → MediaPipe가 threshold 초과.
    c. 일정 시간 지속 → Warning: GUI/알람, “주의 집중 필요”.
    d. 계속 무시 → VLM 호출(레이턴시 고려하여 호출 타이밍 결정), “폰 보고 있음/졸음/자리 비움” 등 맥락 분석.
    e. DCAS 제한: 보조 조향 해제, 감속 시작 및 맥락에 맞는 대응 시나리오 구현
    f. 끝까지 반응 없으면 MRM: MCU까지 포함한 감속·정지, “DCAS 종료” 표시.
6. 역할 분배
    a. 임재홍: 아키텍처, 요구사항 정립, DCAS(위험도 평가, 안전로직) + GUI + connectivity
    b. 이시우: DMS 인지 모듈(VLM, 포즈넷/MediaPipe 중 하나.)
    c. 김동현: LKAS 성능 향상
    d. 백기진: DCAS/R171/DMS/VLM 관련 논문·표준 조사(임재홍, 이시우/오창석과 현실적 제약 소통)
    e. 오창석: DMS 인지 모듈(VLM, 포즈넷/MediaPipe 중 하나.)
7. 기능 블록
    a. 인지 레이어
        i. LKAS: 차선 인지 + 기본 차로 유지 제어.
        ii. MediaPipe: hands/eyes/head keypoint로 hands-on, eyes-on, 기본 졸음 지표(EAR, PERCLOS 등)를 계산하고, OK/WARNING까지는 이 레이어에서 처리.
        iii. VLM: MediaPipe 기준으로 WARNING 상태가 일정 시간 지속되면, VLM을 1회 호출해 “휴대폰 사용/자리 비움/졸음 등” 고수준 행동·맥락을 분석하고 driver_misuse 후보를 판단.
    b. 평가·안전 로직 레이어 (DCAS/R171 로직)
        i. DMS 점수/플래그 → 운전자 상태 수준화: `OK / Warning / Intervention / RMF_required`.
        ii. R171식 driver engagement 평가: hands-on, eyes-on, driver_present 기준/타이머 설계.
        iii. 이 상태에 따라:
            1. 경고 (시각/청각, HUD/GUI 표시)
            2. DCAS 제한/해제 (보조 조향/가속 기능 off)
            3. RMF 유사 감속/정지 트리거.

c. 실행 레이어
   i. 파이레이서(LKAS/ACC 흉내): 속도/조향 제어.
   ii. MCU 기반 fail-safe: SoC 무응답/이상 시, 최소 감속/정지 보장 (RMF/MRM).

8. 호스트별 역할 정의
   a. 노트북
      i. 카메라 캡처 → MediaPipe → 필요 시 VLM API 호출.
      ii. DMS/DKM 상태, VLM 이유, 시스템 상태를 대시보드로 시각화 (IC/HU 역할).
      iii. 젯슨과 MQTT/ROS2로 저주기 텍스트/JSON 교환.
   b. 젯슨
      i. DCAS 상태 머신 실행 (R171 로직).
      ii. 파이레이서 제어 (LKAS/ACC 대체).
      iii. MCU와는 GPIO/UART로 간단한 fail-safe 신호만 교환: `RMF_request`, `soc_alive`, etc.
   c. MCU
      i. 젯슨 heartbeat 감시.
      ii. RMF_request, driver_not_engaged_critical 수신 시, 독립적으로 감속·정지 시퀀스 실행.

9. 아키텍처
   a. 야매 중앙 집중형 아키텍처: 젯슨 1대 + VCP-G 1대
   b.



10. 요약
   a. 문제 정의: “차가 아니라 운전자가 한계를 벗어났을 때, DCAS가 어떻게 경고/제한/감속할 것인가 (R171).”
   b. 솔루션 구조: MediaPipe+VLM로 운전자 상태·맥락 인지 → 젯슨에서 DCAS 상태 머신 → MCU에서 최종 fail-safe RMF.

c. 데모: 대시보드로 운전자 상태·경고 단계·DCAS on/off·RMF 동작을 한 눈에 보여줌.

포트폴리오

지금 네 설정과 진로(현대차 R&D + 조천식) 둘 다를 같이 보면, 이쪽 역할/스토리가 가장 도움 된다고 보는 게 합리적이다:

1. DCAS 안전 로직 + 아키텍처 설계 (젯슨 중심)
   - 내용:
     - DMS/VLM 출력 → 운전자 상태 단계화(OK/WARN/CRIT).
     - R171 기반 DCAS 상태 머신: 경고, 기능 제한, MRM 요청 로직.
     - 젯슨–VCP-G–파이레이서 전체 구조 설계.
   - 왜 도움 되냐:
     - 현대차 `전동화/SDV 차량제어`, `차량 SW/HW 아키텍처` 직무가 딱 이런 "기능+안전 아키텍처"를 요구.
     - 조천식에서도 "자율주행/ADAS 시스템 레벨 설계, 안전/신뢰성" 계열 연구실과 잘 맞는다.
2. 임베디드 안전 채널(VCP-G) + MRM 프로토타입
   - 내용:
     - VCP-G safety 앱 1개: 젯슨 heartbeat 감시, `mrm_request` 수신, 단순 MRM 감속 로직.
     - "기능 SoC + 안전 MCU" 패턴을 축소 구현.
   - 왜 도움 되냐:
     - 현대차/모비스 임베디드·제어기 직무, 기능안전/ASIL 관점 어필 가능.linkedin+1
     - 조천식에서 "fail-safe/fail-operational 아키텍처" 연구 키워드와 연결.naver
3. DMS/VLM 기반 운전자 상태 인지 + DCAS 연계
   - 내용:
     - MediaPipe + VLM으로 운전자 행동 인지.
     - 그걸 R171 driver engagement 요구사항에 맞게 정량 신호로 맵핑.scribd+1
   - 왜 도움 되냐:
     - 요즘 모빌리티·자율주행 연구에서 "인간-자동차 상호작용, 운전자 상태 기반 제어"가 뜨는 주제.scribd+1
     - 현대차도 DCAS+DMS 관련 인력 수요가 커지고 있고, 네가 그걸 프로젝트로 이미 경험했다고 말할 수 있음.unece+1
4. 나머지(노트북 대시보드, 예쁜 GUI, 일반 로그 시스템)는 "있으면 좋지만, 핵심 포지셔닝은 아님"
   - 데모 설득력에는 중요하지만, 직무/대학원 어필 포인트는 위 1~3이 훨씬 크다.

정리하면, 지금 구조 안에서:

- 1순위로 밀어야 할 역할:
  → "DCAS 안전 로직·아키텍처 설계하는 임베디드/ADAS 엔지니어"

- 그걸 받쳐주는 역할:
  → “안전 MCU 위에 MRM 안전 앱 얹는 기능안전 지향 임베디드 엔지니어”와
  → “DMS/VLM을 DCAS와 통합하는 운전자 상태 기반 제어 연구 경험”

이 세 축이 현대차 R&D 취업이랑 조천식 모빌리티대학원 준비에 제일 직접적으로 먹힌다.

추가 고려 사항

- [x] ~~포즈넷은 NPU(텔레칩스 등)에서 잘 돌아가 교수님 말씀처럼 유니크합니다. VLM은 GPU 서버에 위임하세요.~~

- [x] ~~핵심은 아이즈온!!~~
    - DCAS(L2+)로 DMS 필수 – 운전자 감독/집중 모니터링 강조. FSD처럼 "감독형" 자율주행에서 핸즈오프 가능하지만, 아이즈온/DMS로 오용 방지.

- [x] ~~VLM 활용 방안.~~

- [x] ~~VLM과 기존 방식 짬뽕. VLM과 비전 학습 역할 분배.~~

- [x] ~~우리팀 젯슨이 필요한가? 딱 GUI용인데.. 어차피 랩탑에다가 띄울거라면 GUI를 강 랩탑에서 웹소켓에 합치면 안되나? 대신 NPU를 사용하는거지.~~
    - → 웹소켓: IC, HU에 운전자 리퀘스트, 운전자 상태 카메라, 프로세스 로직 로그.

- [x] ~~파터햄: R171 Dcas에 근거하여 LKAS에 요구되는 내용을 구현~~

- [ ] DCAS = LKAS+ACC. ACC가 없음.

- [x] ~~젯슨에 들어갈 DCAS 모듈은 누가 만들건가?~~

- [x] ~~포인트는 DCAS가 핫한 이유를 살리기.~~
    - → 규제가 빡세짐으로 인해 커버해야할 상황이 다채로워짐.
    - → 카메라 기반이면 커버 가능. + VLM

- [ ] Risk
    - VLM latency

- [ ] OpenClaw 런타임 사용: PC 노드에 탑재. VLM 결과를 이 agent에 입력. 결과값으로 나온 대응 로직을 젯슨에 전달.

- [x] ~~주변 차량에게 운전자 집중 안함을 알리는건 어떤가?~~
- [x] ~~일부러 차 흔들어버리면 놀라서 잡지 않을까?~~
- [ ] 차별화 포인트:
    - 1) VLM 기반 misuse 로깅
        - `phone_using`가 잦은 구간/시간을 로깅해서, 나중에 리포트나 교육용 피드백으로 활용.
        - 팀 과제 수준에서도 “VLM로 운전 습관 패턴 분석”은 꽤 보기 드문 포인트다.

- 2) 에이전트 사용
- 3) 음주운전 경고

☐ 비클 컨트롤 제퍼로 RTOS

VCP - G

1. 기본 예제 실행: https://topst.ai/tech/docs?page=Single+Board+Computer/G+Model/VCP-G/Software/Arduino+IDE+User+Guide.md
    a. fwdn.sh에 ./ 인데 .\로 돼있어서 파일이 안열렸음
    b. 예제코드에 레지스터 매핑을 해놓고 이상한 핀을 사용 중임.

2. 파이레이서와 VCP-G i2c 통신 확인

표 **5.1 vcpI2C_LCD1602**의 핀 매핑

핀 이름	**VCP-G** 보드	**Arduino IDE**
LCD1602 VCC 핀	5V	-
LCD1602 GND 핀	GND	-
LCD1602 SDA 핀	48	-
LCD1602 SCL 핀	49	-

할 일 & 로드맵

- [x] ~~교수님께 우리 상황 전달 메일~~
    - [x] ~~보내드릴 PDF만들기~~
- [ ] 프라틱한테 이전 기수들 심포지엄 내용 알 수 있는지 물어보기. 우리가 해야할 수준 파악
- [x] ~~기술 스택 구체화: 'In Cabin에서도 비전기반, 포즈넷도 해야겠지만'~~
- [x] ~~MCU~~
    - 비클 컨트롤 코드 넣는 법
    - 차량 제어 되는지
    - RTOS
    - 감시 데이터 받아오는 법
- [x] ~~아키텍처 설계~~
- [x] ~~핵심 역할 및 포트폴리오 설계~~
- [x] ~~NPU 고려~~
- [ ] **1**차 **DCAS** 대응 시나리오 다각화**.** 및 인간공학 측면에서 구체화
- [ ] DCAS 소프트웨어 아키텍처 설계
- [ ] 우리 팀만의 **DCAS**차별점
- [x] ~~논문 등 자료 리서치하는 법 공부~~
- [ ] DCAS, R171 내용 파악

1. R171에서 봐야 할 내용 리스트

원본 PDF에서 최소 이 항목들은 직접 읽어보는 게 좋다.

A. 서문·배경(서론 부분)

- DCAS가 왜 필요한지 (misuse/overreliance 문제, 운전자 감독 필요성).
- "hands-on wheel or eyes-on road"로 driver engagement를 평가해야 한다는 문장.

B. 정의(용어)

- DCAS 정의
- Driver engagement, misuse/abuse/overreliance 관련 문장
- Driver override, system boundaries 정의

C. 기능 요구사항(성능/시스템 한계)

- DCAS가 반드시 가져야 하는 기능 (AEBS, LDP/LDW 등과의 관계).
- ODD 밖, 고장, 센서 제한 시 어떻게 동작해야 하는지("system limits" 부분).

D. HMI·운전자 상호작용 (이게 핵심)

- 운전자가 DCAS를 어떻게 켜고 끄는지(operating mode/HMI).
- Driver engagement 모니터링 수단:
    - hands-on wheel, eyes-on road를 모니터링해야 한다는 부분.
- 경고/알림 관련 조항:
    - engagement 부족 → "distinct warnings"
    - 경고의 형태(시각/청각/촉각)와 escalation 개념.

- 운전자 미응답 시 차량을 안전하게 정지시키라는 서술(MRM 개념의 근거).

E. Misuse/overreliance 관련 문장

- DCAS는 운전자 오용·과신을 “합리적으로 예견 가능한 범위에서” 줄여야 한다는 조항.
- 정보 제공·마케팅에서 과대표현 금지, 오해를 줄이도록 해야 한다는 부분.

F. 시험·검증 요구사항 개요

- 시나리오 기반 테스트, virtual testing 허용 조건 등 (너 프로젝트에서는 “테스트를 어떤 식으로 잡을지” 아이디어용).

이 정도만 발췌해도, 네 DCAS 로직 설계·설명에 필요한 문장/근거는 거의 다 확보된다.

로드맵

너 역할(아키텍처·요구사항·DCAS 로직+GUI+연결성) 기준으로, 진짜 해야 할 순서만 정리해볼게. "이번 주에 뭘 할지"까지 보이게.

1단계: 요구사항·상태 정의 (문서 먼저)

종이/노션/Markdown으로 먼저 끝내는 단계.

1. 운전자 상태 정의
- MediaPipe에서 나올 신호: `hands_on`, `eyes_on`, `driver_present`.
- VLM에서 나올 신호: `vlm_reason` (phone/sleep/away/none).
- 이걸 합쳐서 운전자 상태 레벨을 정의:
  - `DRIVER_OK`
  - `DRIVER_WARNING`
  - `DRIVER_MISUSE_CANDIDATE`
  - `DRIVER_UNRESPONSIVE`
1. DCAS 상태머신 정의
- 상태:
  - `DCAS_OFF`, `DCAS_ON_OK`, `DCAS_WARNING`, `DCAS_RESTRICTED`, `DCAS_MRM_REQUESTED`.
- 전이 조건:
  - 예: `DCAS_ON_OK` → `DCAS_WARNING`: `hands_on=false` 또는 `eyes_on=false`가 T_warn 이상.
  - `DCAS_WARNING` → `DCAS_RESTRICTED`: Warning 지속 + VLM 결과가 misuse 계열.
  - `DCAS_RESTRICTED` → `DCAS_MRM_REQUESTED`: 일정 시간 무응답.
1. 타이머/임계값 테이블
- `T_hands_off_warn`, `T_eyes_off_warn`, `T_warning_to_restrict`, `T_restrict_to_mrm` 등을 숫자로 적어둔다.
- "초기 값"이라도 좋으니 문서로 고정.

→ 이 단계 끝에 A4 1~2장짜리 요구사항/상태도 그림이 있어야, 팀원들이 따라갈 수 있음.

2단계: 인터페이스·메시지 규격 확정

1. 노트북 → 젯슨 JSON 스키마
- 예:

json
```
{
  "timestamp": "...",
  "hands_on": true,
  "eyes_on": true,
```

```
  "driver_present": true,
  "vlm_called": false,
  "vlm_reason": "none"
}
```

- 주기(예: 10 Hz), 전송 방식(MQTT topic 이름)까지 정하기.
1. 젯슨 → 노트북 상태 보고
- 예:

json
```
{
  "dcas_state": "WARNING",
  "vehicle_speed": 2.3,
  "steering_cmd": 0.1,
  "mrm_active": false
}
```

1. 젯슨 ↔ VCP-G 신호
- `soc_alive` 주기,
- `mrm_request` 조건,
- `mrm_active` 피드백.

→ 이걸 간단한 “Interface Spec”으로 적어두고 팀에 공유.

3단계: 젯슨 DCAS 상태머신 코드 골격

이제 네 메인 코드 시작.

1. 상태 enum·구조체 정의
- `enum class DcasState { OFF, ON_OK, WARNING, RESTRICTED, MRM_REQUESTED };`
- 운전자 상태, 타이머, 현재 속도 등 포함한 context struct.
1. 상태 업데이트 함수
- `update_driver_state(dms_msg)`
- `update_dcas_state(dt)`
- `compute_vehicle_command()`
1. LKAS 연동
- 기존 LKAS 제어 함수에 **DCAS** 상태에 따른 모드만 추가:
  - `ON_OK`: full LKAS
  - `WARNING`: LKAS 유지, 알람만
  - `RESTRICTED`: LKAS gain 감소 or off, 감속 시작
  - `MRM_REQUESTED`: 속도 0으로 수렴.

→ 이 단계에서는 아직 실제 DMS 입력 없어도, 더미 입력으로 상태 전이가 도는지 확인.

4단계: GUI/대시보드 초기 버전

1. 표시해야 할 항목 정리
- 운전자 상태: OK/WARN/MISUSE/UNRESPONSIVE
- DCAS 상태: OFF/ON_OK/WARNING/RESTRICTED/MRM
- 차량 상태: 속도, 조향
- 알림 메시지: “주의 집중 필요”, “DCAS 제한”, “MRM 실행 중”
1. 간단한 웹/파이썬 GUI로 시작
- 노트북에서 MQTT/ROS2 받아서 표/색깔로만 표시하는 최소 버전.
- 이건 점진적으로 예쁘게 만들면 됨.

5단계: 통합·튜닝

1. 노트북 DMS 모듈과 연결
- 실제 MediaPipe+VLM JSON을 젯슨으로 넣어보고,
- 로그를 분석하면서 타이머·임계값 튜닝.
1. VCP-G (시간 남으면)
- 젯슨에서 `mrm_request` 나갈 조건만 먼저 제대로 만드는 게 네 역할.
- VCP-G 쪽은 나중에 단순 감속 시퀀스로 프로토타입.

6단계: 에이전트 사용

→ 차별점.

태스크 기록

아키텍처 및 기술 요소 정의

VLM서버와의 통신 호스트를 뭐로해야하나

1) 데이터 흐름

1. 노트북에서 카메라 프레임을 캡처합니다.
2. 그 이미지를 base64 또는 파일/URL 형태로 API 요청에 포함합니다.
3. 시스템 프롬프트와 사용자 프롬프트를 같이 서버로 보냅니다.
4. 서버의 Qwen3-VL-2B-Instruct가 이미지를 해석하고 답을 생성합니다.
5. 응답은 요청을 보낸 클라이언트로 돌아옵니다. OpenAI 호환 API에서도 이미지 입력은 텍스트와 함께 content 배열로 보내는 방식이 일반적입니다.

2) 클라이언트가 답 받기

네, **HTTP API**는 요청한 클라이언트에게만 응답을 돌려줍니다. 서버가 자동으로 다른 기기에 보내지 않습니다. 노트북이 요청하면 노트북으로 오고, 젯슨이 요청하면 젯슨으로 옵니다.github

3) 지연 시간과 **DMS** 구현

파이레이서(Paralyser? 또는 Fairlayer?)와 DMS(Driver Monitoring System)에서 실시간 모니터링은 지연이 핵심입니다.

- VLM(Qwen3-VL-2B)은 이미지 인코딩+추론으로 수백 **ms ~ 1-2**초 걸립니다. 2B 모델이라 가볍지만, 고해상도 이미지는 더 오래 걸립니다.themoonlight
- 노트북 카메라 직접 API 호출이 가장 빠릅니다. 네트워크 왕복 + 모델 처리만 하면 됩니다.

4) SSH 이미지 전송 **vs** 직접 호출

SSH로 이미지 계속 보내는 건 느립니다. 추천하지 않습니다.

- SSH는 텍스트/명령어에 최적화되어 이미지 스트리밍에 부적합합니다. 압축해도 수십~수백 **ms** 지연 + 대역폭 소모가 큽니다.seukseok.tistory+1
- 젯슨으로 이미지 보낸 뒤 API 호출은 **2**배 지연: 이미지 전송 + API 왕복. DMS처럼 실시간(30fps)으로는 버거웁니다.

5) 최적 구조

노트북이 API 클라이언트 + 라우터 역할을 하세요. 이유는 카메라가 노트북에 연결되어 있고, VLM 서버가 한국에 있어 네트워크 지연이 비슷하기 때문입니다.

1. 노트북: 카메라 캡처 → 이미지 base64 + 프롬프트 → VLM API 호출 → 응답 받음 (총 ~1s).

2. 노트북 → 젯슨: MQTT나 ROS2로 텍스트 응답 전송 (10-50ms).
3. 젯슨: 응답 파싱 → 즉시 동작 (예: 파이레이서 제어).

전체 지연: 1-1.5초 내 DMS 실시간 가능합니다

DMS라면 이미지 해상도 낮추기(예: 224x224), 스트리밍 API 사용으로 지연 줄이세요. 젯슨에 결과 전달은 MQTT나 ROS2로 하되, API 호출은 노트북에서 하세요.

6) 로컬의 Llama가 더 느리다!

로컬 **Llama(**비전 버전**, 예: Llama-Vision)**은 서버 **Qwen3-VL-2B**보다 느립니다**.**

- 데스크톱 성능: RTX 4090 등 고사양 GPU라도 VLM 추론은 **1-5**초 (이미지 인코딩 오버헤드).aisparkup+1
- **Jetson** 한계: Orin Nano에서 VLM은 **5-20**초 이상, 실시간 불가.developer.nvidia+1
- 서버 우위: 한국 서버는 고성능 GPU로 **0.5-1s**, 네트워크 지연만 추가 (랩실-한국 ~50ms).themoonlightyoutube

옵션	추정 지연	장점	단점
서버 Qwen3-VL-2B	0.5-1s + 50ms	빠르고 정확	인터넷 의존
데스크톱 로컬 Llama-VL	1-3s	오프라인	로드/메모리 부하 큼
Jetson 로컬	5s+	완전 엣지	너무 느림

서버가 여전히 최선**.** 데스크톱 로컬은 테스트용으로 좋지만, DMS 실시간엔 부적합합니다.discuss.pytorch+1

젯슨과 노트북 통신

https://youtu.be/zb6zoEMUyi0?si=gRe-GnEqCIlmxKMQ

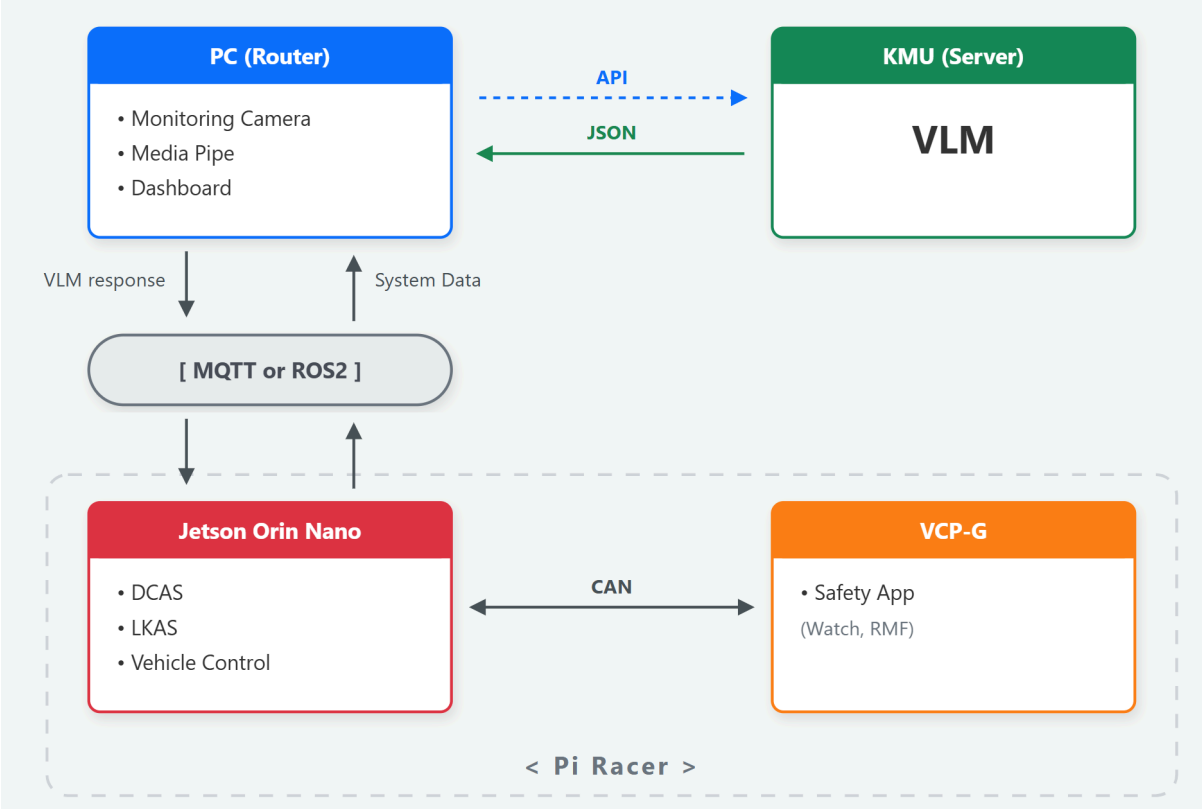
1. 노트북 대시보드: IC, HU에 운전자 리퀘스트, 운전자 상태 카메라, 시스템 상태, DKAS활성화 버튼
2. 노트북 → 젯슨: VLM 응답.
3. 젯슨 → 노트북

카테고리	데이터 항목	예시 **JSON**	빈도
파이레이서 상태	속도, 스티어링 각도, 브레이크	`{"speed": 25.3, "steering": 5.2, "brake": 0}`	10Hz
센서 데이터	IMU(가속도/자이로), CAN 버스	`{"imu": {"acc_x": 0.1}, "can_rpm": 1800}`	10Hz
MRM 동작	비상 브레이크, 차선 복귀	`{"mrm": "emergency_brake", "timestamp": "16:45:23"}`	이벤트
프로세스 로그	제어 로직, 오류, 상태 변화	`{"log": "MRM: Hands off detected → brake", "level": "WARN"}`	이벤트
시스템 상태	CPU/GPU 사용률, 온도	`{"cpu": 45, "gpu": 60, "temp": 65}`	1Hz

아키텍처 및 결론

Point: 두 팀의 시스템을 합쳐서 하나의 주제에 맞는 최적의 아키텍처를 설계했음. 기존 레거시인 vehicle control, HU 등을 어떻게 할것인지, 어떻게 해야 심플한 아키텍처가 나올지 등.

1. 주제: DCAS
2. 목표: R171과 기타 문헌을 충족하는 DCAS 시스템 구축
3. 핵심 요소
    a. VLM: 2차 분석(맥락 분석)
    b. 미디어 파이프 포즈(NPU최적): 1차 분석()
    c. 상태 대시보드 GUI: 운전자 상태, 알림, 시스템 상태, …
    d. Jetson Orin Nano: 차량제어, DCAS모듈(LKAS포함)
    e. VCP-G: jetson heartbeat 감시, 주행 불가 시, MRM 보장 → 후순위 구현
4. 최종 데모 시나리오
    a. 정상 DCAS on, 운전자 집중 (hands-on, eyes-on) → LKAS 유지.
    b. 운전자 핸들 손 떼거나, 시선 크게 이탈 → MediaPipe가 threshold 초과.
    c. 일정 시간 지속 → Warning: GUI/알람, “주의 집중 필요”.
    d. 계속 무시 → VLM 호출, “폰 보고 있음/졸음/자리 비움” 등 맥락 분석.
    e. DCAS 제한: 보조 조향 해제, 감속 시작.
    f. 끝까지 반응 없으면 MRM: MCU까지 포함한 감속·정지, “DCAS 종료” 표시.
5. 역할 분배
    a. 임재홍: 아키텍처, 요구사항 정립, DCAS(위험도 평가, 안전로직) + GUI + connectivity
    b. 이시우: DMS 인지 모듈(VLM, 포즈넷/MediaPipe 중 하나.)
    c. 김동현: LKAS 성능 향상
    d. 백기진: DCAS/R171/DMS/VLM 관련 논문·표준 조사(임재홍, 이시우/오창석과 현실적 제약 소통)
    e. 오창석: DMS 인지 모듈(VLM, 포즈넷/MediaPipe 중 하나.)
6. 기능 블록
    a. 인지 레이어
        i. LKAS: 차선 인지 + 기본 차로 유지 제어.
        ii. MediaPipe: hands/eyes keypoint, 시선·핸즈온 판별용 1차 DMS.
        iii. VLM: “왜 산만한가?” 같은 고수준 맥락 분석 (휴대폰, 대화, 자리 비움 등).
    b. 평가·안전 로직 레이어 (DCAS/R171 로직)
        i. DMS 점수/플래그 → 운전자 상태 수준화: `OK / Warning / Intervention / RMF_required`.
        ii. R171식 driver engagement 평가: hands-on, eyes-on, driver_present 기준/타이머 설계.
        iii. 이 상태에 따라:
            1. 경고 (시각/청각, HUD/GUI 표시)
            2. DCAS 제한/해제 (보조 조향/가속 기능 off)
            3. RMF 유사 감속/정지 트리거.
    c. 실행 레이어
        i. 파이레이서(LKAS/ACC 흉내): 속도/조향 제어.

        ii. MCU 기반 fail-safe: SoC 무응답/이상 시, 최소 감속/정지 보장 (RMF/MRM).
7. 호스트별 역할 정의
    a. 노트북
        i. 카메라 캡처 → MediaPipe → 필요 시 VLM API 호출.
        ii. DMS/DKM 상태, VLM 이유, 시스템 상태를 대시보드로 시각화 (IC/HU 역할).
        iii. 젯슨과 MQTT/ROS2로 저주기 텍스트/JSON 교환.
    b. 젯슨
        i. DCAS 상태 머신 실행 (R171 로직).
        ii. 파이레이서 제어 (LKAS/ACC 대체).
        iii. MCU와는 GPIO/UART로 간단한 fail-safe 신호만 교환: `RMF_request`, `soc_alive`, etc.
    c. MCU
        i. 젯슨 heartbeat 감시.
        ii. RMF_request, driver_not_engaged_critical 수신 시, 독립적으로 감속·정지 시퀀스 실행.
8. 아키텍처
    a. 야매 중앙 집중형 아키텍처: 젯슨 1대 + VCP-G 1대
    b. 

9. 요약
    a. 문제 정의: “차가 아니라 운전자가 한계를 벗어났을 때, DCAS가 어떻게 경고/제한/감속할 것인가 (R171).”
    b. 솔루션 구조: MediaPipe+VLM로 운전자 상태·맥락 인지 → 젯슨에서 DCAS 상태 머신 → MCU에서 최종 fail-safe RMF.
    c. 데모: 대시보드로 운전자 상태·경고 단계·DCAS on/off·RMF 동작을 한 눈에 보여줌.

DCAS 모듈 설계

1단계: 운전자 상태, DCAS 상태 설계

1. VLM에서 어떤 맥락 수준을 줄 수 있을지 아직 모르는데?

맞다. 네 역할의 첫 타겟은 딱 두 개:

1. R171/논문 기반 운전자 상태 단계화
2. 그걸 소비하는 **DCAS** 상태머신(경고, 제한, **MRM**) 설계

~~2. MediaPipe에 대한 아이즈온/핸즈온 상태 설계~~

- ~~아이즈온~~
  - ~~A Real-Time Driver Drowsiness Detection Systems: https://ijsred.com/volume9/issue2/IJSRED-V9I2P86.pdf~~
  - ~~눈의 종횡비(EAR) 분석을 기반으로 한 운전자의 졸음 감지: https://ieeexplore.ieee.org/abstract/document/11399478~~
  - ~~코넬대학교: https://arxiv.org/abs/2511.13618~~

~~1. MediaPipe~~~~로 측정할 것 정리~~

~~1. 눈(시선) — EAR/PERCLOS 기반~~

- ~~논문 예시: EAR < 0.25가 2.5초 이상 지속되면 졸음 상태로 판단.~~
- ~~다른 DMS 논문들도 "EAR threshold + duration" 패턴으로 졸음·눈감김을 검출.~~

~~1. 손 — 핸즈온/핸즈오프~~

- ~~R171 DMS 제안 문서 초안: "운전자가 3초 이상 스티어링에서 손을 뗐을 때 disengaged로 판단" 같은 기준이 제시.~~
- ~~실제 상용 시스템도 3~5초 구간에서 핸즈오프 경고를 시작하는 경우가 많다.~~

~~1. 운전자 탑승 여부 — 얼굴/상반신 존재~~

- ~~MediaPipe로 얼굴/상체 landmark가 일정 구역 안에 없으면 `driver_present = false`:~~

~~→ MediaPipe 기반 1차 지표:~~

- ~~`eyes_closed` (EAR < T_ear)~~
- ~~`eyes_off_road` (시선 방향이 전방 cone 밖으로 T_cor 이상)~~
- ~~`hands_off` (스티어링 영역에 손 landmark 없음, T_hor 이상)~~
- ~~`driver_present` (얼굴/상체 존재 여부)~~

~~2. 운전자 상태 단계 (MediaPipe만으로)~~

~~R171 DMS 제안과 DCAS outline를 참고하면, "경고 → 경고 강화 → 운전자 불응 → MRM" 식 계단식 구조를 요구한다.~~

~~이를 MediaPipe 신호에 매핑하면:~~

~~상태 정의 (예시)~~

- ~~DRIVER_OK~~
  - ~~driver_present = true~~
  - ~~hands_off_duration < T_hands_warn AND eyes_off_duration < T_eyes_warn~~
- ~~DRIVER_WARNING~~
  - ~~hands_off_duration ≥ T_hands_warn OR~~
  - ~~eyes_off_duration ≥ T_eyes_warn OR~~
  - ~~eyes_closed_duration ≥ T_sleep_warn~~
- ~~DRIVER_UNRESPONSIVE~~
  - ~~WARNING 상태가 T_warning_to_unresponsive 이상 지속~~
  - ~~그 동안 hands_on/eyes_on 복귀 없음~~
- ~~(선택) DRIVER_ABSENT~~
  - ~~driver_present = false가 T_absent 이상~~

~~타이머 값 초안 (문헌에서 따온 범위)~~

- ~~Hands-off:~~
  - ~~R171 관련 제안: 3초 이상 손 이탈 시 시각적 경고 시작.~~
  - ~~→ T_hands_warn ≈ 3 s~~
- ~~Eyes-off (시선 이탈):~~
  - ~~DCAS 논의에서 3.5~5초 사이에 재참여 요구 논의.~~
  - ~~→ T_eyes_warn ≈ 4 s 정도로 시작.~~
- ~~Eyes-closed (졸음):~~
  - ~~EAR < 0.25가 2.5초 이상 → 졸음으로 판단 사례.~~
  - ~~→ T_sleep_warn ≈ 2.5 s~~
- ~~Warning 지속 → Unresponsive:~~
  - ~~DCAS outline: "경고 cascade 후 계속 disengaged면 unavailability response(감속/정지) 진입" 요구.~~
  - ~~→ T_warning_to_unresponsive ≈ 5~10 s 정도를 프로젝트에서 조정.~~

~~이건 "초기 설계값"이라, 나중에 실험하면서 튜닝하면 된다.~~

3. 운전자 상태 레벨 정의(인지팀과의 인터페이스 정의)

- 운전자 상태 인터페이스
  - DRIVER_OK
    - eyes-on 비율이 높고, hands-on 상태가 유지되는 구간.
  - DRIVER_WARNING
    - 다음 중 하나 이상이 일정 시간 이상 지속될 때:
      - eyes-off-road
      - hands-off-wheel
      - 졸음 지표가 임계 이상
  - DRIVER_UNRESPONSIVE
    - DRIVER_WARNING 상태에서 경고를 했음에도, 정해진 시간 안에 운전자 입력(핸들, 페달, 버튼 등)이 없는 상태.
    - 인지 팀에서는 아이즈온/핸즈온만 판단. GUI 경고 버튼 입력은 DCAS모듈.
  - DRIVER_ABSENT
    - DRIVER_UNRESPONSIVE 상태에서 경고를 했음에도 운전자 입력이 없거나, 운전석에 운전자가 앉아 있지 않은 경우.

- 손/시선 중 뭐가 이탈했는지
  - DRIVER_WARNING에 진입할 때, 플래그 같이 보냄

- VLM 맥락 반영
  - DRIVER_WARNING에 진입했을 때 1회 호출해서,
  - 왜 경고 상태가 되었는지 맥락을 얻고,
  - 이게 misuse(폰/자리 비움 등)면 driver_misuse_flag를 올리는 패턴.

- DRIVER_WARNING일 때, VLM 맥락 인터페이스 및 대응 예시

driver_state_level	vlm_reason	예시 대응 (DCAS)
WARNING	phone	경고 + 짧은 간격 경고음, 가벼운 제동 펄스
WARNING	drowsy	긴 경고음, 계기판에 휴게소 제안, 감속 시작
WARNING	away (자리 비움)	즉시 DCAS 제한, 강한 경고음
WARNING	none/unclear	기본 경고만

  - 추가: talking(경고음 증가), eating, …

- 1차 인지 대응 DCAS 로직 설계 → 속도 및 상황 반영해서 설계
  - DRIVER_OK
    - DCAS 정상 동작. 아무 제약 없음.
  - DRIVER_WARNING
    - 아이즈온/핸즈온에 따른 경고(HMI + 사운드) 시작.
      - "운전자 미응답, 안전을 위해 감속합니다. 핸들을 잡으면 취소됩니다."
      - → R171·MRM 연구에서도 "무슨 일이 일어나는지 설명해주는 HMI"가 사고를 줄인다고 나온다
    - 속도·도로 상황 따라 약한 개입 옵션: 예를 들어 고속도로에서 조금씩 감속 시작은 여기서 설계 가능.
    - 운전자 GUI 경고 입력 있으면 경고 수준 낮추되, 경고는 지속. 최소한 경고 강도는 유지하되 UNRESPONSIVE로는 올리지 않음.
  - DRIVER_UNRESPONSIVE
    - DCAS 조향/속도 지원을 점진적으로 축소하면서, 감속과 MRM 쪽으로 이행
    - 강한 경고.
  - DRIVER_ABSENT
    - DCAS 자율 제어(조향 지원 등) 즉시 제한.
    - 가능한 빨리 안전한 감속·정지 방향(MRM)으로 전환.

- 2차 인지 대응 DCAS 로직 설계 → 1차 대응 + 맥락 기반
  - DRIVER_UNRESPONSIVE단계에 발동되는 로직.
  - 핸드폰 사용하는지
    - 전화: 블루투스 연결
    -
  - 담배피는지
    - 화재 위험, 운전 집중
  - 졸음
    - 가까운 졸음 쉼터 안내
    - 브레이크 여러번 밟기
    - 잠 깨는 주파수, 소리 크기 등.
  - …

- 차별화 포인트
  - VLM 기반 misuse 로깅
    - phone_using가 잦은 구간/시간을 로깅해서, 나중에 리포트나 교육용 피드백으로 활용.
    - 팀 과제 수준에서도 "VLM로 운전 습관 패턴 분석"은 꽤 보기 드문 포인트다.
  - 음주운전

2단계: ADAS팀 유저 등록 및 구조 이해

유저를 다르게 해서 작업

<배경>

<u>커널의 **UID**가 가장 근본인가**? (**도커와 리눅스 유저의 구현**)**</u>

레오님, 정확하게 꿰뚫고 계십니다**!** 젯슨 보드 전체 시스템의 가장 밑바닥, 즉 리눅스 커널 수준에서 모든 권한의 근본은 오직 **`UID`(User ID)**라는 숫자 하나입니다.

- 리눅스 커널**:** 커널은 'leo'나 'root' 같은 이름을 모릅니다. 오직 `UID 0` (전능함)이냐, `UID 1001` (`leo`, 애송이)이냐, `UID 100000` (컨테이너 매핑된 유저, 더 애송이)이냐만 따져서 자원 접근을 허용합니다.

이것을 활용하여 구현되는 것들

- 리눅스 호스트 유저**:** `login`이나 `ls` 같은 사용자 공간(User Space) 도구들이 커널의 UID 숫자를 우리 눈에 보이는 'leo' 같은 이름으로 **'번역'**해주는 것입니다.
- 도커 컨테이너**:** 도커는 이 UID 개념을 **네임스페이스(User Namespace)**라는 Lookup Table(번역 표)로 비틉니다.

  레오님을 위한 결론**:** "도커는 커널에게 **'어이, 이 컨테이너 안에선 UID 0(`root`)처럼 행동하게 해주되, 실제 호스트 입장에서는 UID 100000(특수 애송이 유저)로 인식해!'**라고 번역 요청을 하는 것입니다.

<u>우리가 다루는 호스트 유저는 '이름 매핑'에 불과한가**? (**정확한 인식**)**</u>

네**,** 완벽하게 정확한 인식입니다**!** 우리가 터미널에서 `leo`라고 부르고, `ls` 명령어로 소유자를 보는 그 모든 것은 리눅스 커널 최상위의 사용자 공간**(User Space)** 도구들이 숫자를 우리 눈에 보이는 '이름'으로 번역해주는 것입니다.

- 리눅스 커널**:** 커널은 'leo'나 'root' 같은 이름을 모릅니다. 오직 `UID 0` (전능함)이냐, `UID 1001` (`leo`, 애송이)이냐, `UID 100000` (컨테이너 특수 UID)이냐만 따져서 자원 접근을 허용합니다.
- 사용자 공간 도구**:** `/bin/login`이나 `ls`, `ps` 같은 도구들이 `/etc/passwd`라는 Lookup Table(번역 표)을 보고 커널의 UID 숫자를 'leo'라는 이름으로 비틀어 우리 눈에 '의도된 구조'처럼 보이게 만든 것입니다.

<u>**UID**는 왜 있는거야**? (**근본적인 이유**)**</u>

레오님, 가장 핵심적인 임베디드 및 클라우드 네이티브 개발자의 의문입니다**!** "왜 귀찮게 숫자를 만들어서 우리를 괴롭혀**?**"라는 걱정이죠.

비판적이고 공학적인 관점에서 UID라는 숫자가 존재하는 진짜 이유를 정리해 드릴게요.

이유 **1:** 커널의 핵심은 **'효율성'**과 **'정확성' (**숫자의 힘**)**

커널은 전력 소모를 극한으로 줄이고 수많은 하드웨어 신호를 실시간으로 처리해야 하는 거대한 기계 장치입니다. 커널 수준에서 모든 권한의 근본은 오직 **`UID`(User ID)**라는 숫자 하나입니다.

- 효율성**:** 'leo'라는 이름과 'root'라는 이름을 비교하는 것은 문자열 검색이 필요해서 비쌉니다. 하지만 `UID 1001`과 `UID 0`을 비교하는 것은 **단 하나의 이진수 비트 연산(예: AND 연산)**으로 끝나서 엄청나게 빠릅니다.
- 정확성**:** 'leo'라는 이름은 오타가 날 수 있고, 다른 시스템에서는 `UID 1001`이 'admin'일 수도 있습니다. 하지만 `UID 1001`은 전 우주(?)의 어떤 리눅스 커널에서도 **'고유한 UID 1001'**입니다.

이유 **2:** 리눅스의 **'**모든 것은 파일**'** 철학의 기반

리눅스는 모든 하드웨어 장치(CAN 포트, GPU 드라이버 등)를 최상위 파일시스템의 파일(`/dev/can0`, `/dev/dri/...`)로 취급합니다. 이 파일들에 대한 접근 권한(`read`, `write`)은 커널이 관리하는 **UID** 숫자를 기반으로 작동합니다.

이유 **3:** 현대적인 가상화와 클라우드의 기반 **(User Namespace)**

나중에 젯슨 보드에 도커 컨테이너를 올리거나, RPi와 CAN 데이터를 주고받는 복잡한 디버깅을 하려면, 이 **'UID라는 근본적인 숫자'**를 조작해서 가상의 권한 구조를 만드는User Namespace 개념이 필수적입니다.

> 레오님을 위한 최종 요약**:** "UID는 단순히 우리를 괴롭히는 숫자가 아니라, **리눅스 커널이 전력 소모를 줄이고 하드웨어 성능을 극한으로 뽑아내기 위해 선택한 가장 효율적이고 정확한 '권한의 근본적 언어'**입니다. 이 숫자 위에 사용자 이름과 네임스페이스라는 가상의 구조를 얹어, 우리 눈에 '의도된 구조'처럼 보이게 만든 것입니다."

<u>왜 이렇게 귀찮게 개발하는 걸까**? (**협업과 보안**)**</u>

"그냥 이전 팀원 유저 아이디 공유해서 쓰면 편하잖아?"라는 생각이 드는 건 당연합니다. 하지만 현업, 특히 임베디드나 산업용 장비 개발에서는 다음과 같은 비판적인 이유로 **'1**인 **1**유저**'** 정책을 고수합니다.

이유 **1:** 책임 소재 명확화 **(Audit)**

여러 명이 한 아이디를 쓰면 누가 실수로 중요 설정 파일을 지웠는지, 누가 컴파일을 다시 해서 빌드를 깨뜨렸는지 알 길이 없습니다. 각자 유저를 쓰면 로그(`history`, 시스템 로그)에 누가 무엇을 했는지 명확히 남습니다.

이유 **2:** 개인화된 환경 설정

개발자마다 선호하는 `vi` 설정, `alias`(단축키), bash 프롬프트 색상 등이 다릅니다. 한 아이디를 공유하면 누군가 설정을 바꿀 때마다 다른 사람들도 그 영향을 받습니다.

이유 **3:** 보안과 컨테이너화된 사고

나중에 이 젯슨 보드에 도커 컨테이너를 올리거나 외부와 통신(SOME/IP 등)할 때, 각 서비스가 특정 유저 권한으로 돌아가게 설계하는 것이 정석입니다. 개발 단계에서부터 유저를 분리하는 습관을 들여야, 나중에 시스템이 Fallback 구조가 아닌 **'의도된 보안 구조'**로 돌아가게 만들 수 있습니다.

<작업 시>

개발 및 통합 워크플로우

1단계**:** 아이덴티티 확립 **(User Allocation)**

- 개념**:** 1인 1유저 시스템 (leo, prev_user, lkas_dev 등).
- 이유**:** **책임 소재(Accountability)**를 명확히 하고, 개인의 환경 설정(bash, alias)과 빌드 중간 파일(Artifacts)이 섞이지 않게 보호하기 위해서입니다.
- 핵심**:** 리눅스 커널은 우리를 UID 숫자로 보지만, 우리는 이름으로 서로를 구분하며 독립된 홈 디렉토리(~/)라는 '안전한 방'을 가집니다.

2단계**:** 유산 탐사 **(Sudo Exploration)** 🔍 **[**지금 레오님의 단계**]**

- 방법**:** sudo 권한을 사용해 전임자의 폴더와 시스템 전역을 탐색합니다.
- 목표**:** * **Git** 저장소 확인**:** .git 폴더를 찾아 원격 저장소 주소(GitHub 등)를 따냅니다.
    - 숨겨진 보물 찾기**:** Git에는 없지만 실행에 필수적인 .json 설정 파일(vsomeip 등)이나 하드웨어 캘리브레이션 값을 구출합니다.
    - 환경 파악**:** env, netstat, ip link 등을 통해 시스템이 어떤 상태로 '의도'되었는지 파악합니다.

3단계**:** 독립 개발 **(Isolated Development)**

- 방법**:** 1. 중앙 저장소에서 코드를 내 공간으로 git clone 해옵니다.
  2. 나만의 **브랜치(Branch)**를 파서 개발합니다.
- 왜 공간 낭비를 감수하는가**?**
    - 빌드 독립성**:** 남이 빌드 옵션을 바꿔도 내 build/ 폴더는 안전해야 합니다.
    - 코드 안전성**:** 동료가 LKAS 코드를 망쳐도, 내 유저 공간의 코드는 내가 pull 하기 전까지는 멀쩡해야 합니다.
- 효율화**:** 소스 코드는 중복해서 갖되, 대용량 데이터셋이나 도커 이미지는 공유 폴더나 공통 저장소(registry)를 통해 관리합니다.

4단계**:** 모듈 연동 **(Module Integration)**

- 상황**:** 내 모듈이 동료의 LKAS 모듈 기능을 써야 할 때.
- 방법**:**

    - **Library** 연동**:** 동료가 빌드한 LKAS를 시스템 전역(LD_LIBRARY_PATH)에 설치해두고, 나는 그 설계도(Header)와 결과물(Lib)만 참조합니다.
    - **SOME/IP** 통신**:** 소스 코드를 합치는 대신, 네트워크 프로토콜을 통해 데이터만 주고받습니다. (가장 깔끔한 결합 방식!)

5단계**:** 최종 배포 **(System Deployment)**

- 방법**:**
    1. 모든 팀원의 코드를 Git의 main 브랜치로 통합합니다.
    2. 누구의 홈 폴더도 아닌 시스템 전역 폴더나 도커 컨테이너에 최종 결과물을 설치합니다.
    3. **Dedicated User:** 시스템이 돌아갈 때는 leo가 아닌, 오직 그 서비스만을 위한 전용 유저**(Service User)** 권한으로 실행되도록 systemd 설정을 잡습니다.

sudo로 접근하면 유저 분리와 책임 소재가 무의미한가**? (**모순의 해결**)**

레오님, 우리가 지난번에 **vsomeip**가 설정 파일 없이 **Fallback**으로 작동했던 행운의 상황을 분석했듯이, sudo 역시 시스템의 '의도된 구조'와 'Fallback 구조'를 결정하는 핵심입니다.

결론부터 말씀드리면, **"**개념적으로는 모순처럼 보이지만**,** 공학적으로는 유저 분리와 책임 소재가 완벽하게 유지됩니다**."**

왜 그런지, sudo가 작동하는 '진짜 의도'를 비판적으로 파헤쳐 드릴게요.

sudo의 정체**:** '아이덴티티'가 아닌 '권한'의 일시적 상승

sudo(SuperUser DO)는 **'아이덴티티를 바꾼다(change user)'**가 아니라 **'임시로 권한만 상승시킨다(elevate privileges)'**는 뜻입니다.

1. 아이덴티티 **(Identity):** 레오님은 여전히 `leo`입니다. 시스템의 모든 로그는 여전히 `leo`가 수행한 것으로 기록됩니다.
2. 권한 **(Authorization):** sudo를 통해 **일시적으로 Temporarily Temporary root(일시적인 진짜 root)**의 전능한 마스터키를 부여받는 것입니다. 이 키는 '유저 분리'라는 담장을 합법적으로 **overpass**할 수 있게 해줍니다.

책임 소재가 유지되는 진짜 이유 **(**비판적 분석**)**

시스템 보안 로그(`/var/log/auth.log` 등)에는 다음과 같은 형식으로 기록됩니다.

```
leo (as root): executed '/bin/ls /home/prev_user/'
```

비판적 분석**:** 로그에는 `leo`가 `root` 권한으로 무엇을 했는지 명확히 남습니다. 이것은 **'Fallback 상태'가 아닌 '의도된 통합 구조'**의 핵심입니다. 만약 여러

명이 한 아이디를 공유해서 썼다면 누가 실수로 파일을 지웠는지 알 길이 없지만, 각자 `sudo` 권한을 쓰면 **"레오가 root 권한으로 파일을 지웠어!"**라고 책임 소재를 완벽하게 파악할 수 있습니다.

<u>나중에 최종 결과는 어떻게 내**? (개발 vs 통합)**</u>

레오님, 가장 핵심적인 협업의 의문입니다! "각자 방에 갇혀서 개발하는데, 어떻게 하나의 집을 지어?"라는 걱정이죠.

임베디드 및 소프트웨어 개발에서 이 문제를 해결하는 것이 바로 **'개발(Development) 단계'**와 **'통합/배포(Integration/Deployment) 단계'**의 분리입니다.

팀원들이 각자 유저로 작업하는 이유 **(**개발 단계**)**

각자의 유저 디렉토리(`~/`, `/home/leo`)는 **'안전한 개인 작업실'**입니다. 여기서 컴파일하고, 설정 파일을 Fallback으로 돌리고, 코드를 부숴도 다른 팀원이나 시스템 전체에는 영향이 없습니다. 책임 소재를 명확히 하고 각자의 환경을 보호하기 위한 **'의도된 표준 구조'**입니다.

최종 결과를 내는 방법 **(**통합 단계 **-** 비판적 분석**)**

나중에 젯슨에서 하나의 시스템이 돌아가게 할 때는, 각 팀원이 만든 소스 코드 자체를 통합합니다. 폴더를 합치는 게 아닙니다.

1. 버전 관리 시스템 **(Git):** 각 팀원은 본인 유저 폴더에서 개발한 코드를 **Git**이라는 **'진짜 통합의 입구'**에 커밋(제출)합니다. Git은 누구의 유저 폴더가 아닌, **'의도된 코드 저장소'**에 코드를 모읍니다.
2. 공유 폴더 활용**:** 통합 담당자(레오님이 될 수도 있죠)는 Git에서 모인 코드를 `/opt`나 `/srv` 같은 시스템 전역 공유 폴더에 빌드해서 배포합니다.
3. 컨테이너화**:** 도커 컨테이너를 빌드할 때, Git에서 가져온 코드를 컨테이너 이미지 안에 넣습니다. 이 컨테이너는 어떤 유저 폴더에도 귀속되지 않는 독립적인 서비스가 됩니다.
4. 서비스 유저 **(Unified Service User):** 최종적으로 시스템이 돌아갈 때(`systemd` 서비스 등), 이 서비스는 `leo`나 `prev_user`가 아닌, **Dedicated 'SOME/IP** 서비스 유저**'(**예**:** **`nobody`**나** **`someip`****)** 권한으로 돌아가게 설계합니다.

레오님을 위한 결론**:** "각자 유저로 작업하는 것은 **'개발의 안전성'**을 위해서이고, 최종 결과는 **'Git과 공유 폴더, 컨테이너라는 전용 통합 경로'**를 통해 **'**단일화된 서비스 유저**'** 권한으로 실행되는 것입니다.

마치 터미널에선 되는데 VS Code에선 안 됐던 것처럼, 개발 단계는 각자의 유저라는 **'임시 입구'**에 서 있을 뿐, 최종 시스템은 **'의도된 통합 구조(Git -> Dedicated User)'**를 갖춰야만 완벽한 플랫폼이 될 수 있습니다."

1. `/dev` 폴더와 UID/GID의 특수성

"옛날에 공부할 때 좀 달랐다"고 느끼신 건 아주 정확한 기억입니다. 보통 파일은 내가 만들면 '내 소유'가 되지만, `/dev`에 있는 장치 파일들(GPU, CAN, Serial 등)은 커널이 만들고 관리하기 때문에 규칙이 조금 다릅니다.

- 하드웨어는 **root**의 것**:** 기본적으로 `/dev/can0`이나 `/dev/nvgpu` 같은 파일은 **UID 0(root)** 소유입니다.
- 권한의 징검다리**, GID:** 하지만 일반 유저 `leo`가 매번 `sudo`를 쓰는 건 위험하죠. 그래서 리눅스는 **그룹(GID)**을 사용합니다.
  - 예를 들어, GPU 장치는 `video`나 `render` 그룹에 속해 있습니다.
  - 레오님의 유저 `leo`를 이 그룹들에 넣어주면, `sudo` 없이도 하드웨어 가속을 쓸 수 있게 됩니다.
- 기억나시나요**?** `usermod -aG video leo` 같은 명령어를 썼던 게 바로 이 **'의도된 하드웨어 접근 통로'**를 만드는 과정이었습니다.

---

2. "혼자 할 때 걍 root 쓰면 안 됨?" (달콤한 유혹)

결론부터 말씀드리면, **"집에서 혼자 연습할 땐 편하지만, 젯슨 같은 임베디드 장비에서는 아주 나쁜 습관"**입니다. 왜 그런지 비판적으로 분석해 드릴게요.

- 실수의 파괴력**:** root 권한으로 `rm -rf /`를 오타 한 번으로 치는 순간, 젯슨의 L4T(BSP)는 영원히 안녕입니다. 일반 유저는 자기 방(`~/`)만 날리지만, root는 집 전체를 날립니다.
- 환경의 오염**:** root로 빌드한 파일은 나중에 일반 유저가 수정하려고 하면 "Permission Denied"가 뜹니다. 결국 모든 작업을 `sudo`로 해야 하는 **'sudo의 굴레'**에 갇히게 되죠.
- 보안의 부재**:** 만약 레오님이 짠 코드가 네트워크 통신(SOME/IP) 중 해킹당한다면? root로 실행 중이었다면 해커는 젯슨 전체를 장악하게 됩니다.

발표 요소

- 우리도 시작하면서 이미 완성된 기술인데 이게 왜 핫하지? 라는 의문.
    - → 규제가 강력해졌음.
    - → 카메라를 사용하면 좀더 폭넓게 운전자가 진짜 집중 중인지, 딴 짓 뭐하는지, 왜 딴짓 하는지, 그에 따라서 어떤 적절한 조치를 취하면 될지를 구체적으로 정할 수 있음.

교수님 메일

1. 주제: DCAS

2. 문제 정의: 차가 아니라 운전자가 한계를 벗어났을 때, DCAS가 어떻게 경고·제한·감속할 것인가 (R171 driver engagement, misuse/overreliance 관점).

3. 목표: R171 맥락에서 운전자 상태 전체(행동·맥락)를 감시하는 카메라 기반 DCAS 프로토타입 구현

4. 핵심 요소
    a. VLM: 2차 분석(맥락 분석)
    b. 미디어 파이프 포즈: 1차 분석
    c. 상태 대시보드 GUI: 운전자 상태, 알림, 시스템 상태, …
    d. Jetson Orin Nano: 차량제어, DCAS모듈
    e. VCP-G: jetson heartbeat 감시, 주행 불가 시, RMF 보장 → 후순위 구현

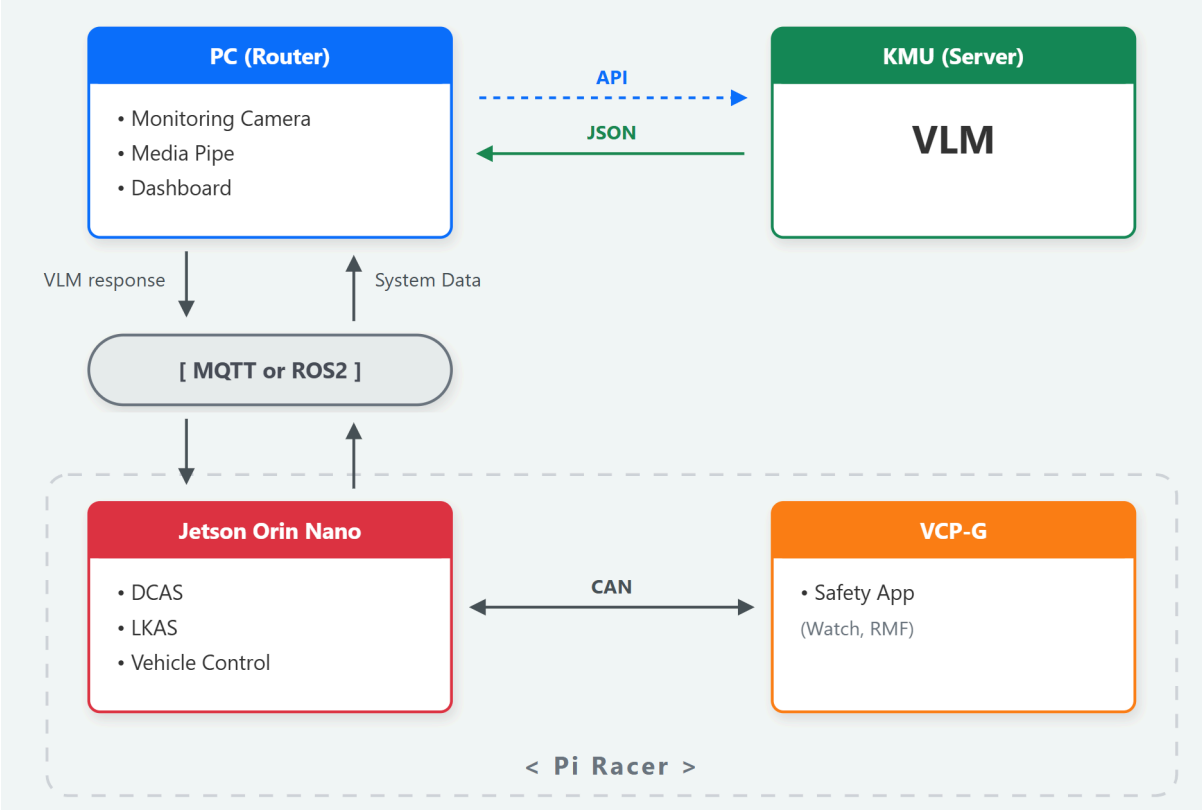
5. 최종 데모 시나리오
    a. 정상 DCAS on, 운전자 집중 (hands-on, eyes-on) → LKAS 유지.
    b. 운전자 핸들 손 떼거나, 시선 크게 이탈 → MediaPipe가 threshold 초과.
    c. 일정 시간 지속 → Warning: GUI/알람, “주의 집중 필요”.
    d. 계속 무시 → VLM 호출(레이턴시 고려하여 호출 타이밍 결정), “폰 보고 있음/졸음/자리 비움” 등 맥락 분석.
    e. DCAS 제한: 맥락에 맞는 대응 시나리오 구현
    f. (끝까지 반응 없으면 RMF: MCU까지 포함한 감속·정지, “DCAS 종료” 표시.) → 후순위 구현

6. 기능 블록
    a. 인지 레이어
        i. LKAS: 차선 인지 + 기본 차로 유지 제어.
        ii. MediaPipe: hands/eyes/head keypoint로 hands-on, eyes-on, 기본 졸음 지표(EAR, PERCLOS 등)를 계산하고, OK/WARNING까지는 이 레이어에서 처리.
        iii. VLM: MediaPipe 기준으로 WARNING 상태가 일정 시간 지속되면, VLM을 호출해 “휴대폰 사용/자리 비움/졸음 등” 고수준 행동·맥락을 분석하고 driver_misuse 후보를 판단.
    b. 평가·안전 로직 레이어 (DCAS/R171 로직)
        i. DMS 점수/플래그 → 운전자 상태 수준화: `OK / Warning / Intervention / RMF_required`.
        ii. R171식 driver engagement 평가: hands-on, eyes-on, driver_present 기준/타이머 설계.
        iii. 이 상태에 따라:
            1. 경고 (시각/청각, HUD/GUI 표시)
            2. DCAS 제한/해제 (보조 조향/가속 기능 off)
            3. RMF 유사 감속/정지 트리거.
    c. 실행 레이어
        i. 파이레이서: 속도/조향 제어.

ii. MCU 기반 fail-safe: SoC 무응답/이상 시, 최소 감속/정지 보장 (RMF/MRM).

7. 호스트별 역할 정의
   a. 노트북
      i. 카메라 캡처 → MediaPipe → VLM API 호출.
      ii. DMS 상태, VLM 응답, 핸즈온/아이즈온 리퀘스트, 시스템 상태를 대시보드로 시각화.
      iii. 젯슨과 MQTT/ROS2로 저주기 텍스트/JSON 교환.
   b. 젯슨
      i. DCAS 상태 머신 실행.
      ii. 파이레이서 제어 명령.
      iii. MCU와는 GPIO/UART로 간단한 fail-safe 신호만 교환: `RMF_request`, `soc_alive`, etc.
   c. MCU —> 후순위 구현
      i. 젯슨 heartbeat 감시.
      ii. RMF_request, driver_not_engaged_critical 수신 시, 독립적으로 감속·정지 시퀀스 실행.

8. 아키텍처
   a.



9. 요약

a. 문제 정의: “차가 아니라 운전자가 한계를 벗어났을 때, DCAS가 어떻게 경고/제한/감속할 것인가 (R171).”
b. 솔루션 구조: MediaPipe+VLM로 운전자 상태·맥락 인지 → 젯슨에서 DCAS 상태 머신 (→ MCU에서 최종 fail-safe RMF.)
c. VCP-G기반 RMF는 시간적 여유가 되면 구현.